

Google Colab Setup

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
import os
GOOGLE_DRIVE_PATH_AFTER_MYDRIVE = 'CV CW'
GOOGLE_DRIVE_PATH = os.path.join('drive', 'My Drive', GOOGLE_DRIVE_PATH_AFTER_MYDRIVE)
print(os.listdir(GOOGLE_DRIVE_PATH))
```

 ['train', 'test', 'vgg_finetuned_mask_classifier.pth', 'Coursework.ipynb', 'sift_bovw_svm_model.pkl', 'hog_svm_model.pkl', 'WIN_20250420_12_47_20_Pro.mp4', 'CourseworkCV.ipynb']

```
import cv2
import numpy as np
from glob import glob
import random
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from skimage import color, img_as_ubyte
from sklearn.cluster import MiniBatchKMeans
from collections import Counter
from sklearn import svm, metrics
from sklearn.metrics import classification_report, accuracy_score
import tensorflow as tf
from PIL import Image
```

```
%matplotlib inline
```

Load the images and labels

```
train_images_path = os.path.join(GOOGLE_DRIVE_PATH, "train", "images") # path to training set images
test_images_path = os.path.join(GOOGLE_DRIVE_PATH, "test", "images") # path to testing set
```

```
train_labels_folder = os.path.join(GOOGLE_DRIVE_PATH, "train", "labels") # train labels
test_labels_folder = os.path.join(GOOGLE_DRIVE_PATH, "test", "labels") # test labels
```

```
# Loading labels from text files and storing in a dictionary
def load_labels_from_folder(labels_folder):
    labels_dict = {}
    # Looping through each label in the folder
    for file in os.listdir(labels_folder):
        file_path = os.path.join(labels_folder, file) # get full path of each label file

        if file.endswith(".txt"): # Ensure a text file and ignores others
            with open(file_path, "r") as f: # Opens in read mode
```

```

label = f.read().strip() # Removes extra unwanted spaces
labels_dict[file.replace(".txt", ".jpeg")] = int(label) # Links filename and label as .txt is replaced with .jpeg
return labels_dict

train_labels = load_labels_from_folder(train_labels_folder)
test_labels = load_labels_from_folder(test_labels_folder)

# Verifying if everything is looking okay
print("Training Labels:", list(train_labels.items())[:5])
print("Testing Labels:", list(test_labels.items())[:5])

↩ Training Labels: [('image_2216.jpeg', 1), ('image_1327.jpeg', 1), ('image_2202.jpeg', 1), ('image_1247.jpeg', 1), ('image_1496.jpeg', 1)]
Testing Labels: [('image_2684.jpeg', 1), ('image_2479.jpeg', 1), ('image_2492.jpeg', 1), ('image_2690.jpeg', 1), ('image_2732.jpeg', 1)]

```

Image processing

```

# Loading images and mathcing with labels (Raw images and labels)
def load_raw_images(images_path, labels_dict):
    X = []
    y = []

    image_files = sorted(glob(os.path.join(images_path, "*.jpeg")))

    for img_path in image_files:
        img_name = os.path.basename(img_path) # Get image file name

        if img_name in labels_dict: # Ensure all images have a label
            img = cv2.imread(img_path)

            X.append(img)
            y.append(labels_dict[img_name])
    # X to hold images and y to hold labels
    return X, y

# Using the paths and function to load the images as 'raw' (i.e no processing done)
X_train_raw, y_train_raw = load_raw_images(train_images_path, train_labels)
X_test_raw, y_test_raw = load_raw_images(test_images_path, test_labels)

fig, axes = plt.subplots(1, 20, figsize=(20, 5))
raw_images = random.sample(X_train_raw, 20)

for ax, img in zip(axes, raw_images):
    ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # BGR to RGB for matplotlib
    ax.axis("off")
plt.show()

```



```
for i in range(5):
    print(f"Shape Image {i+1}: {raw_images[i].shape}") # Checking shape (to determine if resizing or any other changes might be necessary for some models)
```

➡ Shape Image 1: (24, 26, 3)
 Shape Image 2: (38, 30, 3)
 Shape Image 3: (84, 73, 3)
 Shape Image 4: (41, 35, 3)
 Shape Image 5: (28, 35, 3)

From the images loaded, resizing, normalisation and conversion will be needed for some of the models that would be experimented with

✓ SIFT(BoVW) + SVM

The feature extraction pipeline was based on the example provided in Lab 6 of the INM460 Computer Vision module at City University of London.

```
# Splitting the raw (i.e unprocessed images) for training and validation using SIFT, BoVW and SVM
X_train_sift, X_val_sift, y_train_sift, y_val_sift = train_test_split(X_train_raw, y_train_raw, test_size=0.2, random_state=42, stratify=y_train_raw)

# Initiate SIFT detector
sift = cv2.SIFT_create()

# Create empty lists for feature descriptors and labels
des_list = []
y_train_list = []

fig, ax = plt.subplots(1, 8, figsize=(10, 8), sharey=True)

# Initialize a list to keep track of valid indices
valid_indices = []

for i in range(len(X_train_sift)): # Using raw images for SIFT
    # Convert image to grayscale (SIFT works on grayscale images)
    img_gray = img_as_ubyte(color.rgb2gray(X_train_sift[i]))

    # Detect keypoints and compute descriptors with SIFT
    kp, des = sift.detectAndCompute(img_gray, None)

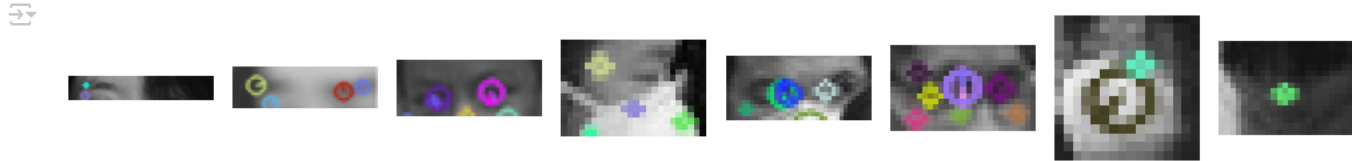
    # Show results for first 4 images
    if i < 8:
        img_with_SIFT = cv2.drawKeypoints(img_gray, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
        ax[i].imshow(img_with_SIFT, cmap="gray")
        ax[i].set_axis_off()

    # Append descriptors and labels (only if descriptors exist)
    if des is not None:
        des_list.append(des)
        # Instead of using the original y_train_sift, append from the filtered list
        y_train_list.append(y_train_sift[i])
        valid_indices.append(i) # Store the index of the valid image

fig.tight_layout()
```

```
plt.show()
```

```
# Convert to array for easier handling (for BoVW step)
des_array = np.vstack(des_list)
```



We can observe some blur in the images above which may cause the performance from SIFT to degrade. In addition, when using SIFT to extract features from the training set; some images might not have had any detectable keypoints (which were then not included in the hist_list). This led to a mismatch of [1894, 1915] and was causing issues when applying the SVM classifier.

```
# Number of centroids/codewords: good rule of thumb is 10*num_classes
k = len(np.unique(y_train_sift)) * 10
```

```
# Use MiniBatchKMeans for faster computation and lower memory usage
batch_size = des_array.shape[0] // 4
kmeans = MiniBatchKMeans(n_clusters=k, batch_size=batch_size, n_init='auto').fit(des_array)
```

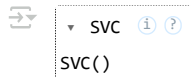
```
# Convert descriptors into histograms of codewords for each image
hist_list = []
idx_list = []
```

```
for des in des_list:
    hist = np.zeros(k)

    idx = kmeans.predict(des)
    idx_list.append(idx)
    for j in idx:
        hist[j] = hist[j] + (1 / len(des))
    hist_list.append(hist)
```

```
hist_array_sift1 = np.vstack(hist_list)
```

```
# Create a classifier: a support vector classifier
classifier = svm.SVC(kernel='rbf')
# We learn the digits on the first half of the digits
classifier.fit(hist_array_sift1, y_train_list)
```



```
hist_list = []
```

```
for i in range(len(X_val_sift)):
    img = img_as_ubyte(color.rgb2gray(X_val_sift[i]))
```

```
kp, des = sift.detectAndCompute(img, None)

if des is not None:
    hist = np.zeros(k)

    idx = kmeans.predict(des)

    for j in idx:
        hist[j] = hist[j] + (1 / len(des))

    # hist = scale.transform(hist.reshape(1, -1))
    hist_list.append(hist)

else:
    hist_list.append(None)

# Remove potential cases of images with no descriptors
idx_not_empty = [i for i, x in enumerate(hist_list) if x is not None]
hist_list = [hist_list[i] for i in idx_not_empty]
y_val_sift = [y_val_sift[i] for i in idx_not_empty]
hist_array = np.vstack(hist_list)

y_pred_sift1 = classifier.predict(hist_array).tolist()

# Plot of predicted vs actual labels
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
for i in range(10):
    idx = np.random.randint(0, len(X_val_sift))
    ax[i].imshow(X_val_sift[idx])
    ax[i].set_title(f'Label: {y_val_sift[i]} \n Prediction: {y_pred_sift1[i]}')
    ax[i].set_axis_off()
fig.tight_layout()
plt.show()
```



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 1
Prediction: 1



Label: 0
Prediction: 1



Label: 1
Prediction: 1



```
print(f"""Classification report for classifier {classifier}:
      {metrics.classification_report(y_val_sift, y_pred_sift1)}\n""")
```



Classification report for classifier SVC():

	precision	recall	f1-score	support
0	0.68	0.17	0.28	75
1	0.83	0.99	0.90	384
2	0.00	0.00	0.00	16
accuracy			0.83	475
macro avg	0.51	0.39	0.39	475
weighted avg	0.78	0.83	0.77	475

```
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zer
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zer
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zer
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Using class weight 'balanced' since presence of class imbalances

```
classifier_bal = svm.SVC(kernel='rbf', class_weight= 'balanced')
classifier_bal.fit(hist_array_sift1, y_train_list)

y_pred_sift1_bal = classifier_bal.predict(hist_array).tolist()

print(f"""Classification report for classifier {classifier}:
      {metrics.classification_report(y_val_sift, y_pred_sift1_bal)}\n""")
```

```

Classification report for classifier SVC():
              precision    recall  f1-score   support

     0       0.36       0.44       0.40        75
     1       0.88       0.70       0.78       384
     2       0.08       0.38       0.13        16

 accuracy          0.65       0.475
 macro avg          0.44       0.51       0.43       475
 weighted avg       0.77       0.65       0.70       475

```

Using class weighting (SVM class weight parameter) improves recall across the minority classes and reduce overfitting of class 1.

Since the model struggled to predict the minority class (mask not worn properly), the images relating to that label would be adjusted by applying filtering and the class imbalance is addressed. A relatively high overall accuracy is observed but extremely low recall on minor classes (0 and 2). The model is biased towards the prediction of class 1 (mask worn) with really high recall.

✓ Data Augmentation (Oversample minority classes)

We used a similar method as proposed by:

Okesha Karunarathne. (2023, October 24). *Handling Class Imbalance in Image Classification: Techniques and Best Practices*. Medium.

Available at: <https://medium.com/@okeshakarunarathne/handling-class-imbalance-in-image-classification-techniques-and-best-practices-c539214440b0>

(Accessed: March 19, 2025)

```

# Import necessary libraries
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation settings (rotation, horizontal and vertical shifts, changes in brightness)
datagen = ImageDataGenerator(rotation_range=20, width_shift_range=0.2, height_shift_range=0.2,
                             brightness_range=[0.7, 1.3], shear_range=0.15, fill_mode='reflect')

# Extract minority class images & labels
X_minority_0 = [img for img, label in zip(X_train_sift, y_train_sift) if label == 0]
y_minority_0 = [0] * len(X_minority_0)

X_minority_2 = [img for img, label in zip(X_train_sift, y_train_sift) if label == 2]
y_minority_2 = [2] * len(X_minority_2)

# Applying the filters through a loop rather than using flow() like above
# Function to generate augmented samples and store
def generate_augmented_samples(images, labels, n_samples):
    augmented_images = []
    augmented_labels = []

    for img, label in zip(images, labels):
        img_array = np.expand_dims(img, axis=0) # Add batch dimension

        # Generate augmented versions of a particular image
        aug_iter = datagen.flow(img_array, np.array([label]),

```

```

        batch_size=1, shuffle=False)

    for _ in range(n_samples):
        aug_img = next(aug_iter)[0][0] # Get the augmented image
        augmented_images.append(aug_img)
        augmented_labels.append(label)

    return augmented_images, augmented_labels

# Generate augmented samples for each minority class depending on imbalances
n_samples_per_img_class0 = 1 # Doubling class 0
n_samples_per_img_class2 = 1 # We are doubling the number of images for class 2 despite more severe imbalances

aug_images_0, aug_labels_0 = generate_augmented_samples(X_minority_0, y_minority_0, n_samples_per_img_class0)
aug_images_2, aug_labels_2 = generate_augmented_samples(X_minority_2, y_minority_2, n_samples_per_img_class2)

# Combining aug data with original data
if isinstance(X_train_sift, list):
    X_train_sift_aug = X_train_sift + aug_images_0 + aug_images_2
    y_train_sift_aug = list(y_train_sift) + aug_labels_0 + aug_labels_2
# Converting to lists since np arrays of different sizes cannot be combined together
else:
    X_train_sift_list = [X_train_sift[i] for i in range(len(X_train_sift))]
    X_train_sift_aug = X_train_sift_list + aug_images_0 + aug_images_2
    y_train_sift_aug = list(y_train_sift) + aug_labels_0 + aug_labels_2

```

✓ SIFT (BoVW) + SVM with Augmented Data

BEST MODEL FROM SIFT+SVM

```

# Create empty lists for feature descriptors and labels
des_list_aug = []
y_train_list_aug = []

fig, ax = plt.subplots(1, 8, figsize=(10, 8), sharey=True)

# Initialize a list to keep track of valid indices
valid_indices_aug = []

for i in range(len(X_train_sift_aug)): # Using raw images for SIFT
    # Convert image to grayscale (SIFT works on grayscale images)
    img_gray = img_as_ubyte(color.rgb2gray(X_train_sift_aug[i].astype(np.uint8)))

    # Detect keypoints and compute descriptors with SIFT
    kp, des = sift.detectAndCompute(img_gray, None)

    # Show results for first 4 images
    if i < 8:
        img_with_SIFT = cv2.drawKeypoints(img_gray, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
        ax[i].imshow(img_with_SIFT, cmap="gray")
        ax[i].set_axis_off()

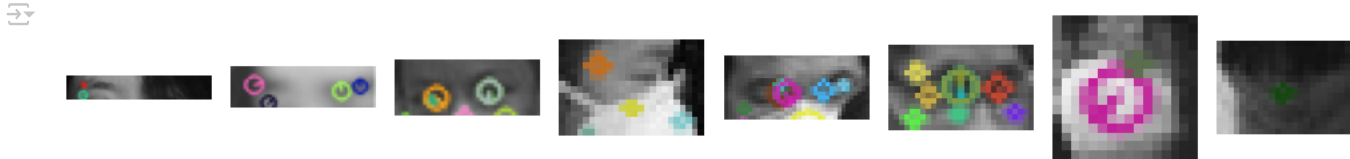
```



```
# Append descriptors and labels (only if descriptors exist)
if des is not None:
    des_list_aug.append(des)
    # Instead of using the original y_train_sift, append from the filtered list
    y_train_list_aug.append(y_train_sift_aug[i])
    valid_indices_aug.append(i) # Store the index of the valid image
```

```
fig.tight_layout()
plt.show()
```

```
# Convert to array for easier handling (for BoVW step)
des_array = np.vstack(des_list_aug)
```



```
# Number of centroids/codewords: good rule of thumb is 10*num_classes
k = len(np.unique(y_train_sift_aug)) * 10
```

```
# Use MiniBatchKMeans for faster computation and lower memory usage
batch_size = des_array.shape[0] // 4
kmeans = MiniBatchKMeans(n_clusters=k, batch_size=batch_size, n_init='auto').fit(des_array)
```

```
# Convert descriptors into histograms of codewords for each image
hist_list = []
idx_list = []
```

```
for des in des_list_aug:
    hist = np.zeros(k)

    idx = kmeans.predict(des)
    idx_list.append(idx)
    for j in idx:
        hist[j] = hist[j] + (1 / len(des))
    hist_list.append(hist)

hist_array_sift1_aug = np.vstack(hist_list)
```

```
# Create a classifier: a support vector classifier
classifier_aug = svm.SVC(kernel='rbf', class_weight='balanced')
```

```
# We learn the digits on the first half of the digits
classifier_aug.fit(hist_array_sift1_aug, y_train_list_aug)
```

```
SVC
SVC(class_weight='balanced')
```

```

hist_list_aug = []

for i in range(len(X_val_sift)):
    img = img_as_ubyte(color.rgb2gray(X_val_sift[i]))
    kp, des = sift.detectAndCompute(img, None)

    if des is not None:
        hist = np.zeros(k)

        idx = kmeans.predict(des)

        for j in idx:
            hist[j] = hist[j] + (1 / len(des))

        # hist = scale.transform(hist.reshape(1, -1))
        hist_list_aug.append(hist)
        # Append the corresponding label only when a histogram is created
        # Ensures hist_list_aug & y_val_sift_aug have same no. of elements
    else:
        hist_list_aug.append(None) # Keep track of images with no descriptors
        # Don't append the label if there is no histogram for the image

# Remove potential cases of images with no descriptors
idx_not_empty_aug = [i for i, x in enumerate(hist_list_aug) if x is not None]
hist_list_aug = [hist_list_aug[i] for i in idx_not_empty_aug]
# Filter y_val_sift based on non-None histograms.
y_val_sift_aug = [y_val_sift[i] for i in range(len(hist_list_aug)) if hist_list_aug[i] is not None]
hist_array_aug = np.vstack(hist_list_aug)

y_pred_sift1_aug = classifier_aug.predict(hist_array_aug).tolist()

fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True) # add random
ax = axes.ravel()

for i in range(10):
    idx = np.random.randint(0, len(X_val_sift))
    img = X_val_sift[idx]
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    ax[i].imshow(img_rgb)
    ax[i].set_title(f'Label: {y_val_sift[i]} \n Prediction: {y_pred_sift1_aug[i]}')
    ax[i].set_axis_off()
fig.tight_layout()
plt.show()

```



Label: 1
Prediction: 1



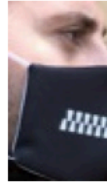
Label: 1
Prediction: 1



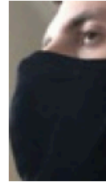
Label: 1
Prediction: 1



Label: 1
Prediction: 2



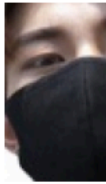
Label: 1
Prediction: 2



Label: 1
Prediction: 1



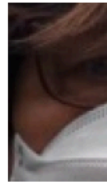
Label: 1
Prediction: 0



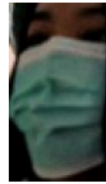
Label: 1
Prediction: 1



Label: 0
Prediction: 0



Label: 1
Prediction: 0



```
print(f"""\nClassification report for classifier {classifier_aug}:\n{metrics.classification_report(y_val_sift, y_pred_sift1_aug)}\n""")
```



```
Classification report for classifier SVC(class_weight='balanced'):
```

	precision	recall	f1-score	support
0	0.36	0.51	0.42	75
1	0.90	0.67	0.77	384
2	0.06	0.31	0.10	16
accuracy			0.63	475
macro avg	0.44	0.50	0.43	475
weighted avg	0.79	0.63	0.69	475

Increasing the number of samples in the minority classes improved the model's performance slightly

We will attempt to optimise the base model and the model with augmented data through grid search

```
import joblib
```

```
# Save the SIFT + BoVW + SVM model (uncomment the below to save again if need be)
# model_path = "/content/drive/MyDrive/CV CW/sift_bovw_svm_model.pkl"
# joblib.dump(classifier_aug, model_path)
# print(f"SIFT + BoVW + SVM model saved at {model_path}")
```



```
SIFT + BoVW + SVM model saved at /content/drive/MyDrive/CV CW/sift_bovw_svm_model.pkl
```

✓ SIFT (BoVW) + SVM Hyperparameter Tuning

Grid Search

```
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['auto', 0.001, 0.01, 0.1, 1],
    'kernel': ['rbf', 'linear', 'poly'],
    'degree': [2, 3] # Only relevant for poly kernel
}

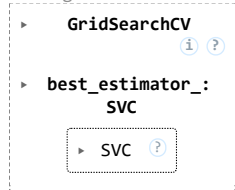
SVM = svm.SVC(class_weight='balanced', random_state=42)

# Defining grid search
grid_search = GridSearchCV(
    estimator=SVM,
    param_grid=param_grid,
    cv=5, # Using 5-fold cross-validation on training set
    scoring= ['accuracy', 'f1_macro'],
    refit = 'f1_macro', # using f1-score to select best model since imbalance dataset
    n_jobs=-1,
    verbose=2,
)
```

✓ Base model optimisation

```
grid_search.fit(hist_array_sift1, y_train_list)
```

↗ Fitting 5 folds for each of 120 candidates, totalling 600 fits



```
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
print("Best f1-score:", grid_search.best_score_)
```

↗ Best parameters: {'C': 100, 'degree': 2, 'gamma': 1, 'kernel': 'rbf'}
 Best accuracy: 0.7191034607921151
 Best f1-score: 0.4690573059746425

```
classifier_opt = grid_search.best_estimator_
y_pred_optsift1_bal = classifier_opt.predict(hist_array).tolist()
```

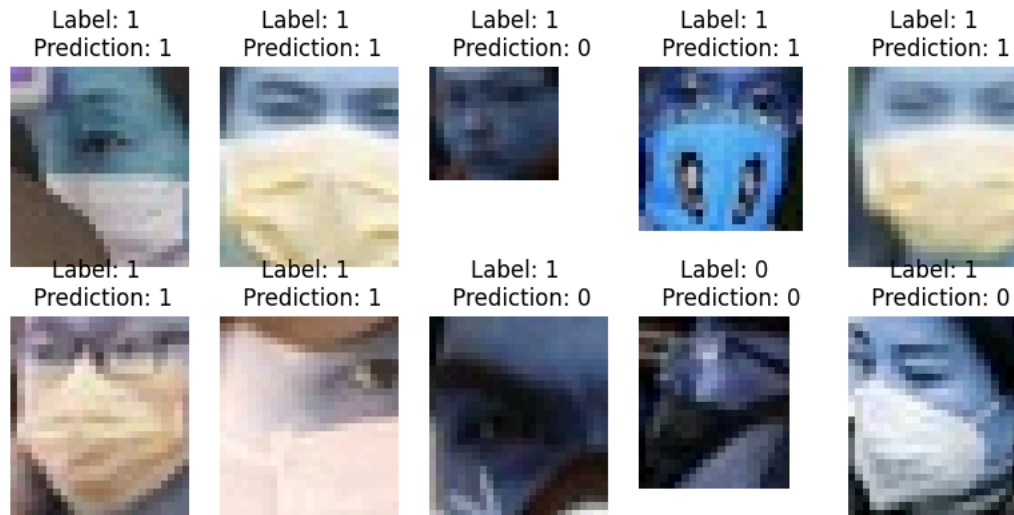
```
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
```

```
for i in range(10):
```

```

idx = np.random.randint(0, len(X_val_sift))
ax[i].imshow(X_val_sift[idx])
ax[i].set_title(f'Label: {y_val_sift[i]} \n Prediction: {y_pred_optsift1_bal[i]}')
ax[i].set_axis_off()
fig.tight_layout()
plt.show()

```



```

print(f"""Classification report for optimised SVM classifier {classifier}:
{metrics.classification_report(y_val_sift, y_pred_optsift1_bal)}\n""")

```



```

Classification report for optimised SVM classifier SVC():
precision    recall  f1-score   support

      0       0.35      0.49      0.41         75
      1       0.88      0.76      0.82        384
      2       0.06      0.12      0.08         16

 accuracy          0.70         475
 macro avg         0.43         475
 weighted avg      0.77         475

```

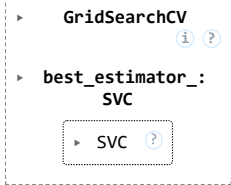
Model with augmented Data optimisation

```

grid_search.fit(hist_array_sift1_aug, y_train_list_aug)

```

↳ Fitting 5 folds for each of 120 candidates, totalling 600 fits



```
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
print("Best f1-score:", grid_search.best_score_)
```

↳ Best parameters: {'C': 100, 'degree': 2, 'gamma': 1, 'kernel': 'rbf'}
 Best accuracy: 0.6663562453806356
 Best f1-score: 0.520190602932377

```
classifier_aug_opt = grid_search.best_estimator_
y_pred_aug_optsift1_bal = classifier_aug_opt.predict(hist_array_aug).tolist()
```

```
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
```

```
for i in range(10):
    idx = np.random.randint(0, len(X_val_sift))
    ax[i].imshow(X_val_sift[idx])
    ax[i].set_title(f'Label: {y_val_sift[i]} \n Prediction: {y_pred_aug_optsift1_bal[i]}')
    ax[i].set_axis_off()
fig.tight_layout()
plt.show()
```

```
print(f"""Classification report for optimised SVM classifier (augmented) {classifier}:
{metrics.classification_report(y_val_sift, y_pred_aug_optsift1_bal)}\n""")
```



Classification report for optimised SVM classifier (augmented) SVC():

	precision	recall	f1-score	support
0	0.38	0.56	0.45	75
1	0.89	0.72	0.80	384
2	0.00	0.00	0.00	16
accuracy			0.67	475
macro avg	0.42	0.43	0.42	475
weighted avg	0.78	0.67	0.71	475

0	0.38	0.56	0.45	75
1	0.89	0.72	0.80	384
2	0.00	0.00	0.00	16
accuracy			0.67	475
macro avg	0.42	0.43	0.42	475
weighted avg	0.78	0.67	0.71	475

Results from the optimised models were unexpectedly very poor. **The best model from SIFT+SVM is the one where we oversampled the minority classes.** Despite a decrease in accuracy, we observed an increase in recall but precision is still extremely low.

SIFT as a feature extractor would be most suitable for distinctive local features and keypoints. Mask detection requires analysing global facial patterns. SIFT also struggles to capture the differences between class 2 and the other classes.

In the next section, we explore **HOG** which is expected to be better at detecting mask. The fixed spatial grid approach would be well-suited for analyzing facial features and their coverage, providing consistent representation of how masks alter facial appearance.

✓ HOG + SVM/MLP

```
from skimage.color import rgb2gray
from skimage.transform import resize
from skimage.feature import hog
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.neural_network import MLPClassifier
```

```

from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE

# Processing images to be suitable for HOG feature extraction
def preprocess_hog(images, image_size=(128, 128)):
    processed = []

    for img in images:
        # BGR to RGB
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        gray = rgb2gray(img_rgb) # Converting to grayscale
        resized = resize(gray, image_size) # Resize

        processed.append(resized)

    return processed

# Function for HOG feature extraction
def hog_features(processed_images):
    hog_features = []

    for image in processed_images:
        features = hog(image,
            orientations=9,
            pixels_per_cell=(8, 8), # Size of each cell in pixel
            cells_per_block=(2, 2), # Group of cells for normalisation
            block_norm='L2-Hys', # Normalisationmethod
            visualize=False # Not generating image
        )
        hog_features.append(features)

    return np.array(hog_features)

# Split training data into 80% train and 20% validation
X_train_split, X_val_split, y_train_split, y_val = train_test_split(
    X_train_raw, y_train_raw, test_size=0.2, stratify=y_train_raw, random_state=42)

# Preprocess images
X_train_processed = preprocess_hog(X_train_split, image_size=(128, 128))
X_val_processed = preprocess_hog(X_val_split, image_size=(128, 128))

# Extract HOG features
X_train_hog = hog_features(X_train_processed)
X_val_hog = hog_features(X_val_processed)

```

▼ Base SVM and MLP Models

```

# Standardize features
scaler = StandardScaler()

```



```
X_train_hog_scaled = scaler.fit_transform(X_train_hog)
X_val_hog_scaled = scaler.transform(X_val_hog)

# Base MLP with HOG feature
mlp = MLPClassifier(random_state=42)
mlp.fit(X_train_hog_scaled, y_train_split)

y_train_pred = mlp.predict(X_train_hog_scaled)
train_accuracy = accuracy_score(y_train_split, y_train_pred)

y_val_pred = mlp.predict(X_val_hog_scaled)
val_accuracy = accuracy_score(y_val, y_val_pred)

print(f"Training Accuracy: {train_accuracy:.4f}")
print(f"Validation Accuracy: {val_accuracy:.4f}")
print("\nClassification report for MLP Base Model:")
print(classification_report(y_val, y_val_pred))
```

↗ Training Accuracy: 1.0000
Validation Accuracy: 0.8309

Classification report for MLP Base Model:				
	precision	recall	f1-score	support
0	0.56	0.47	0.51	75
1	0.88	0.93	0.91	388
2	0.29	0.12	0.17	16
accuracy			0.83	479
macro avg	0.57	0.51	0.53	479
weighted avg	0.81	0.83	0.82	479

```
model = svm.SVC(kernel='rbf', class_weight='balanced')
model.fit(X_train_hog_scaled, y_train_split)
y_train_pred = model.predict(X_train_hog_scaled)
y_val_pred = model.predict(X_val_hog_scaled)
print("Training Accuracy:", accuracy_score(y_train_split, y_train_pred))
print("Validation Accuracy:", accuracy_score(y_val, y_val_pred))
print(classification_report(y_val, y_val_pred))
```

↗ Training Accuracy: 0.9932114882506528
Validation Accuracy: 0.8475991649269311

	precision	recall	f1-score	support
0	0.58	0.51	0.54	75
1	0.89	0.95	0.92	388
2	0.00	0.00	0.00	16
accuracy			0.85	479
macro avg	0.49	0.49	0.49	479
weighted avg	0.81	0.85	0.83	479

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control. Please see the documentation for more details.

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control. Please see the documentation for more details.

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control. Please see the documentation for more details.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

From the base model, we observed severe overfitting and prediction of the minority classes is underperforming. We will:

- 1. Attempt to optimise the models through Grid Search (will add early stopping for MLP)
- 2. If no improvement is observed, we will oversample the minority classes

Grid Search

✓ MLP

```
param_grid = {
    'hidden_layer_sizes': [(100,), (100, 50)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate_init': [0.001, 0.01],
    'max_iter': [500],
    'early_stopping': [True] # Early stopping to prevent overfitting and stop model training when validation score is not improving
}
```

```
mlp = MLPClassifier(random_state=42)
```

```
grid_search = GridSearchCV(
    mlp,
    param_grid,
    cv=3, # Fewer folds for speed
    scoring=['accuracy', 'recall_macro'], # Track both accuracy and recall_macro
    refit='recall_macro', # Select best model based on recall_macro
    n_jobs=-1 # Parallelize
)
```

```
grid_search.fit(X_train_hog_scaled, y_train_split)
```



```
# Print the best parameters and score
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
print("Best recall:", grid_search.best_score_)
```

```

➦ Best parameters: {'activation': 'tanh', 'alpha': 0.0001, 'early_stopping': True, 'hidden_layer_sizes': (100,), 'learning_rate_init': 0.01, 'max_iter': 500, 'solver': 'sgd'}
Best accuracy: 0.5937127466996336
Best recall: 0.6253162439933031

```

```

# Evaluate the best model on the validation set
mlp_best = grid_search.best_estimator_
y_val_pred = mlp_best.predict(X_val_hog_scaled)
y_train_pred = mlp_best.predict(X_train_hog_scaled)
train_accuracy = accuracy_score(y_train_split, y_train_pred)

```

```

print(f"Training accuracy: {train_accuracy:.4f}")
print("\nClassification report for validation set:")
print(classification_report(y_val, y_val_pred))

```

```

➦ Training accuracy: 0.9655

```

```

Classification report for validation set:

```

	precision	recall	f1-score	support
0	0.40	0.69	0.51	75
1	0.93	0.76	0.84	388
2	0.13	0.25	0.17	16
accuracy			0.73	479
macro avg	0.49	0.57	0.51	479
weighted avg	0.82	0.73	0.76	479

SVM

```

# Define parameter grid

```

```

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.0001, 0.001, 0.01],
    'kernel': ['linear', 'rbf']
}

```

```

SVM = svm.SVC(class_weight='balanced', random_state=42)

```

```

# Define grid search

```

```

grid_search = GridSearchCV(
    estimator=SVM,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation on training set
    scoring=['accuracy', 'recall_macro'], # Track both accuracy and recall_macro
    refit='recall_macro', # Select best model based on recall_macro
    n_jobs=-1,
    verbose=2
)

```

```

grid_search.fit(X_train_hog_scaled, y_train_split)

```

↳ Fitting 5 folds for each of 24 candidates, totalling 120 fits

▸ GridSearchCV

▸ best_estimator_: SVC

▸ SVC

```
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
print("Best recall:", grid_search.best_score_)
```

↳ Best parameters: {'C': 0.1, 'gamma': 0.0001, 'kernel': 'rbf'}
Best accuracy: 0.6793733681462141
Best recall: 0.5712151042373572

```
# Get best model
best_model = grid_search.best_estimator_
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Recall (Macro):", grid_search.best_score_)
print("Best Cross-Validation Accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
```

```
# Evaluate on training set
y_train_pred = best_model.predict(X_train_hog_scaled)
train_accuracy = accuracy_score(y_train_split, y_train_pred)
print("\nTraining Accuracy:", train_accuracy)
```

```
# Evaluate on validation set
y_val_pred = best_model.predict(X_val_hog_scaled)
val_accuracy = accuracy_score(y_val, y_val_pred)
print("Validation Accuracy:", val_accuracy)
print("Validation Classification Report:")
print(classification_report(y_val, y_val_pred))
```

↳ Best Parameters: {'C': 0.1, 'gamma': 0.0001, 'kernel': 'rbf'}
Best Cross-Validation Recall (Macro): 0.5712151042373572
Best Cross-Validation Accuracy: 0.6793733681462141

Training Accuracy: 0.77911227154047
Validation Accuracy: 0.7139874739039666
Validation Classification Report:

	precision	recall	f1-score	support
0	0.37	0.92	0.52	75
1	0.96	0.69	0.80	388
2	0.42	0.31	0.36	16
accuracy			0.71	479
macro avg	0.58	0.64	0.56	479
weighted avg	0.85	0.71	0.75	479

A decrease in accuracy from 85% to 71% is observed. However, recall has improved and this is to be expected since Grid Search was done to identify the model with best performing recall. The precision for Class 0 has decrease but recall has increased significantly.

✓ Using SMOTE to oversample

The idea is to oversample the features extracted by HOG for the minority classes (0 and 2)

```
Counter(y_train_split)
```

```
↗ Counter({1: 1552, 0: 301, 2: 62})
```

```
# Define SMOTE
```

```
smote = SMOTE(
    sampling_strategy={0: 600, 1: 1552, 2: 124}, # Target counts
    k_neighbors=3, # Use 3 neighbors for class 2's small size
    random_state=42
)
```

```
# Apply SMOTE to HOG features
```

```
X_train_hog_scaled_resampled, y_train_split_resampled = smote.fit_resample(X_train_hog_scaled, y_train_split)
print("New training set size:", X_train_hog_scaled_resampled.shape)
print("New class distribution:", Counter(y_train_split_resampled))
```

```
↗ New training set size: (2276, 8100)
New class distribution: Counter({1: 1552, 0: 600, 2: 124})
```

```
# Training SVM with tuned parameters
```

```
model = svm.SVC(C=0.1, gamma=0.0001, kernel='rbf', class_weight='balanced', random_state=42)
model.fit(X_train_hog_scaled_resampled, y_train_split_resampled)
```

```
↗ SVC
SVC(C=0.1, class_weight='balanced', gamma=0.0001, random_state=42)
```

```
# Evaluate on training set
```

```
y_train_pred = model.predict(X_train_hog_scaled_resampled)
train_accuracy = accuracy_score(y_train_split_resampled, y_train_pred)
print("Training Accuracy:", train_accuracy)
```

```
# Evaluate on validation set
```

```
y_val_pred = model.predict(X_val_hog_scaled)
val_accuracy = accuracy_score(y_val, y_val_pred)
print("Validation Accuracy:", val_accuracy)
print("Validation Classification Report:")
print(classification_report(y_val, y_val_pred))
```

```
↗ Training Accuracy: 0.8971880492091389
Validation Accuracy: 0.8413361169102297
Validation Classification Report:
precision    recall  f1-score   support
```

0	0.89	0.23	0.36	75
1	0.84	0.99	0.91	388
2	0.00	0.00	0.00	16
accuracy			0.84	479
macro avg	0.58	0.41	0.42	479
weighted avg	0.82	0.84	0.79	479

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

✓ Using TensorFlow Datasets to oversample

```

# Import necessary libraries
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# Data augmentation settings (rotation, horizontal and vertical shifts, changes in brightness)
datagen = ImageDataGenerator(rotation_range=20, width_shift_range=0.2, height_shift_range=0.2,
                             brightness_range=[0.7, 1.3], shear_range=0.15, fill_mode='reflect')

# Extract minority class images & labels
X_minority_0 = [img for img, label in zip(X_train_split, y_train_split) if label == 0]
y_minority_0 = [0] * len(X_minority_0)

X_minority_2 = [img for img, label in zip(X_train_split, y_train_split) if label == 2]
y_minority_2 = [2] * len(X_minority_2)

# Applying the filters through a loop rather than using flow() like above
# Function to generate augmented samples and store
def generate_augmented_samples(images, labels, n_samples):
    augmented_images = []
    augmented_labels = []

    for img, label in zip(images, labels):
        img_array = np.expand_dims(img, axis=0) # Add batch dimension

        # Generate augmented versions of a particular image
        aug_iter = datagen.flow(img_array, np.array([label]),
                                batch_size=1, shuffle=False)

        for _ in range(n_samples):
            aug_img = next(aug_iter)[0][0] # Get the augmented image
            augmented_images.append(aug_img)
            augmented_labels.append(label)

    return augmented_images, augmented_labels

# Generate augmented samples for each minority class depending on imbalances
n_samples_per_img_class0 = 1 # Doubling class 0
n_samples_per_img_class2 = 1 # We are doubling the number of images for class 2 despite more severe imbalances

```

```

aug_images_0, aug_labels_0 = generate_augmented_samples(X_minority_0, y_minority_0, n_samples_per_img_class0)
aug_images_2, aug_labels_2 = generate_augmented_samples(X_minority_2, y_minority_2, n_samples_per_img_class2)

# Combining aug data with original data
if isinstance(X_train_split, list):
    X_train_split_aug = X_train_split + aug_images_0 + aug_images_2
    y_train_split_aug = list(y_train_split) + aug_labels_0 + aug_labels_2
# Converting to lists since np arrays of different sizes cannot be combined together
else:
    X_train_split_list = [X_train_split[i] for i in range(len(X_train_split))]
    X_train_split_aug = X_train_split_list + aug_images_0 + aug_images_2
    y_train_split_aug = list(y_train_split) + aug_labels_0 + aug_labels_2

# Preprocess images
X_train_aug_processed = preprocess_hog(X_train_split_aug, image_size=(128, 128))
# Extract HOG features
X_train_aug_hog = hog_features(X_train_aug_processed)
X_train_hog_aug_scaled = scaler.fit_transform(X_train_aug_hog)

```

✓ MLP with Augmented Data and Tuned Hyperparameters

Initializing the model with the best parameters

```

best_mlp = MLPClassifier(
    activation='tanh',
    alpha=0.0001,
    early_stopping=True,
    hidden_layer_sizes=(100,),
    learning_rate_init=0.01,
    max_iter=500,
    solver='sgd',
    random_state=42
)

```

Train the model on your augmented HOG features

```
best_mlp.fit(X_train_hog_aug_scaled, y_train_split_aug)
```



MLPClassifier



```

MLPClassifier(activation='tanh', early_stopping=True, learning_rate_init=0.01,
              max_iter=500, random_state=42, solver='sgd')

```

```
y_val_pred = best_mlp.predict(X_val_hog_scaled)
```

Calculate training accuracy

```
y_train_pred = best_mlp.predict(X_train_hog_aug_scaled)
```

```
train_accuracy = accuracy_score(y_train_split_aug, y_train_pred)
```

```
print(f"Training accuracy: {train_accuracy:.4f}")
```

```
print("\nClassification report for validation set:")
```

```
print(classification_report(y_val, y_val_pred))
```

↗ Training accuracy: 0.9789

Classification report for validation set:

	precision	recall	f1-score	support
0	0.34	0.85	0.48	75
1	0.96	0.66	0.78	388
2	0.12	0.19	0.15	16
accuracy			0.67	479
macro avg	0.47	0.57	0.47	479
weighted avg	0.83	0.67	0.71	479

✓ SVM with Augmented Data and Tuned Hyperparameters

```
# Training SVM with tuned parameters and augmented data
model = svm.SVC(C=0.1, gamma=0.0001, kernel='rbf', class_weight='balanced', random_state=42)
model.fit(X_train_hog_aug_scaled, y_train_split_aug)
```

↗

▼ SVC ⓘ ?

SVC(C=0.1, class_weight='balanced', gamma=0.0001, random_state=42)

```
# Evaluate on training set
y_train_pred = model.predict(X_train_hog_aug_scaled)
train_accuracy = accuracy_score(y_train_split_aug, y_train_pred)
print("Training Accuracy:", train_accuracy)

# Evaluate on validation set
# Preprocess images
X_val_processed_aug = preprocess_hog(X_val_split, image_size=(128, 128))
# Extract HOG features
X_val_hog_aug = hog_features(X_val_processed_aug)
X_val_hog_aug_scaled = scaler.transform(X_val_hog_aug) #Using transform and not fit_transform

y_val_pred = model.predict(X_val_hog_aug_scaled) # Predict on the validation set
val_accuracy = accuracy_score(y_val, y_val_pred)
print("Validation Accuracy:", val_accuracy)
print("Validation Classification Report:")
print(classification_report(y_val, y_val_pred))
```

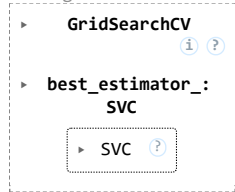
↗ Training Accuracy: 0.7884108867427568
Validation Accuracy: 0.6951983298538622
Validation Classification Report:

	precision	recall	f1-score	support
0	0.36	0.77	0.49	75
1	0.94	0.70	0.80	388
2	0.16	0.31	0.21	16
accuracy			0.70	479
macro avg	0.49	0.59	0.50	479
weighted avg	0.82	0.70	0.73	479

✓ BEST MODEL FROM HOG+SVM

```
grid_search.fit(X_train_hog_aug_scaled, y_train_split_aug)
```

↳ Fitting 5 folds for each of 24 candidates, totalling 120 fits



```
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_])
print("Best recall:", grid_search.best_score_)
```

↳ Best parameters: {'C': 1, 'gamma': 0.0001, 'kernel': 'rbf'}
 Best accuracy: 0.8147484094852515
 Best recall: 0.5824425113169773

```
# Evaluate on training set
y_train_pred = best_model.predict(X_train_hog_aug_scaled)
train_accuracy = accuracy_score(y_train_split_aug, y_train_pred)
print("Training Accuracy:", train_accuracy)
```

```
# Evaluate on validation set
y_val_pred = best_model.predict(X_val_hog_scaled)
val_accuracy = accuracy_score(y_val, y_val_pred)
print("Validation Accuracy:", val_accuracy)
print("\nValidation Classification Report:")
print(classification_report(y_val, y_val_pred))
```

↳ Training Accuracy: 0.8020193151887621
 Validation Accuracy: 0.7139874739039666

```
Validation Classification Report:
              precision    recall  f1-score   support

     0       0.37         0.92         0.52         75
     1       0.96         0.69         0.80        388
     2       0.42         0.31         0.36         16

 accuracy          0.71         0.71         0.71         479
 macro avg         0.58         0.64         0.56         479
 weighted avg         0.85         0.71         0.75         479
```

```
#hog_model_path = "/content/drive/MyDrive/CV CW/hog_svm_model.pkl"
#joblib.dump(best_model, hog_model_path)
#print(f"HOG + SVM model saved at {hog_model_path}")
```

📁 HOG + SVM model saved at /content/drive/MyDrive/CV CW/hog_svm_model.pkl

For the base models ran, SVM failed to predict class 2 while we observed some success for MLP. We attempted to optimise the model where we observed a decrease in accuracy but improvement in recall (this is due to grid search being done to optimise recall due to imbalanced data). SMOTE was not successful, hence we implemented oversampling of the images of the minority classes. Since the model struggles to predict both class 0 and class 2, we doubled the number of images from each of those two classes. The best performing classifier is the SVM one where the model outperformed MLP in all the metrics. In the case of MLP, we also observed overfitting which we tried to mitigate through early stopping.

✓ CNN

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
import random
from torchvision import models, transforms
from torch.optim import lr_scheduler
import time
import copy

# We defined this for consistency in our results but this did not help
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)

# Dataset preparation/wrapping
# To prepare the dataset to be used with PyTorch's Dataloader and implement
# transformations for training and validation
class maskdataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels
        self.transform = transform

    def __len__(self): # define the number of samples in the dataset
        return len(self.images)

    def __getitem__(self, idx):
        image = cv2.cvtColor(self.images[idx], cv2.COLOR_BGR2RGB) #cv2 uses BGR
        image = Image.fromarray(image)
```

```

    if self.transform:
        image = self.transform(image)

    return image, self.labels[idx]

```

CNN Model Training set up with Transfer Learning (**MobileNetV3**)

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # use GPU if available
print(device)

```

→ cuda

```

# Data preparation section
def data_processing(X_train_raw, y_train_raw):
    # Splitting training data into train and validation (80:20)
    X_train, X_val, y_train, y_val = train_test_split(X_train_raw, y_train_raw, test_size=0.2, stratify=y_train_raw, random_state=42)

    # Data transformations required for MobileNetV3 (224x224 images and Normalized)
    train_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(), # some random tranformations for better generalizability
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    # Only Resizing and Normalizing for Validation data
    val_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    # Datasets and Dataloaders
    train_dataset = maskdataset(X_train, y_train, transform=train_transform)
    val_dataset = maskdataset(X_val, y_val, transform=val_transform)
    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

    return train_loader, val_loader

# Looking at weight distribution for each class because of imbalances
# this is done so that we can adjust class weight in the loss function to penalize misclassification of minority classes more severely.
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)
print(f"Class distribution: {class_counts}")
print(f"Class weights: {class_weights}")

→ Class distribution: [ 376 1940   78]
   Class weights: [ 2.12234043  0.41134021 10.23076923]

```

The main training function was based of the example provided in Lab 8 of the INM460 Computer Vision module at City University of London.

```

# Main Training function
def train_mobilenetv3(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15):
    model = model.to(device)
    # Track best model
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    # Time tracking
    since = time.time()

    # Training metrics history
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_acc': [],
        'best_epoch': 0
    }

    # Main training loop going over specified number of epochs
    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")
        print('-' * 20)

        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # Set model to training mode
                dataloader = train_loader
            else:
                model.eval() # Set model to evaluation mode
                dataloader = val_loader

            running_loss = 0.0 # Total loss for the running epoch
            running_corrects = 0 # Correct predictions
            all_preds = [] # For classification performance metrics
            all_labels = []

            # Iterate over data
            for inputs, labels in dataloader:
                inputs, labels = inputs.to(device), labels.to(device)

                # Zero the parameter gradients
                optimizer.zero_grad()

                # Forward pass
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                # Backward + optimize only if in training phase
                if phase == 'train':
                    loss.backward()
                    optimizer.step()

            # Statistics

```

```

running_loss += loss.item() * inputs.size(0)
running_corrects += torch.sum(preds == labels.data)

# Collect predictions and labels for metrics
if phase == 'val':
    all_preds.extend(preds.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())

# Calculate epoch statistics after each epoch
dataset_size = len(dataloader.dataset)
epoch_loss = running_loss / dataset_size # average loss
epoch_acc = running_corrects.double() / dataset_size # average accuracy

print(f"{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}")

# Deep copy the model with best accuracy on validation set
if phase == 'val':
    history['val_loss'].append(epoch_loss)
    history['val_acc'].append(epoch_acc.item())

    if epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())
        history['best_epoch'] = epoch

# Print detailed metrics for the last epoch or when improved
if epoch == num_epochs - 1 or epoch_acc > best_acc:
    print(classification_report(all_labels, all_preds, target_names=['0', '1', '2']))
elif phase == 'train':
    history['train_loss'].append(epoch_loss)

# Update learning rate scheduler based on validation accuracy when performance plateaus
if scheduler is not None and phase == 'val':
    scheduler.step(epoch_acc)

print()

# Training complete
time_elapsed = time.time() - since
print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")
print(f"Best val acc: {best_acc:.4f} (epoch {history['best_epoch'] + 1})")

# Load best model weights
model.load_state_dict(best_model_wts)
return model, history

```

To train the model, we are using CE loss initially with adjusted class weights; giving more importance to class 2 compared to the other classes to address the imbalances and improve on the model sensitivity to underrepresented class.

```

# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (freezing layers and amend fully connected layer for our labels)
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

```

```
# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)
# scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
# Train model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

 Downloading: "https://download.pytorch.org/models/mobilenet_v3_large-5c1a4163.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v3_large-5c1a4163.pth
 100%|██████████| 21.1M/21.1M [00:00<00:00, 112MB/s]
 Epoch 1/15

 train Loss: 0.9246 Acc: 0.6585
 val Loss: 0.7949 Acc: 0.7349

 Epoch 2/15

 train Loss: 0.6828 Acc: 0.7661
 val Loss: 0.7085 Acc: 0.8455

 Epoch 3/15

 train Loss: 0.5735 Acc: 0.8319
 val Loss: 0.7304 Acc: 0.7432

 Epoch 4/15

 train Loss: 0.4461 Acc: 0.8292
 val Loss: 0.7151 Acc: 0.6180

 Epoch 5/15

 train Loss: 0.3442 Acc: 0.8679
 val Loss: 0.5882 Acc: 0.8142

 Epoch 6/15

 train Loss: 0.2515 Acc: 0.9091
 val Loss: 0.6038 Acc: 0.8747

 Epoch 7/15

 train Loss: 0.2216 Acc: 0.9149
 val Loss: 0.5888 Acc: 0.8768

```

Epoch 8/15
-----
train Loss: 0.2314 Acc: 0.9044
val Loss: 0.6346 Acc: 0.8956

Epoch 9/15
-----
train Loss: 0.2482 Acc: 0.9180
val Loss: 0.5638 Acc: 0.8643

Epoch 10/15
-----
train Loss: 0.2245 Acc: 0.9170
val Loss: 0.6334 Acc: 0.8998

Epoch 11/15
-----
train Loss: 0.2339 Acc: 0.9170
val Loss: 0.5999 Acc: 0.8643

```

- With 30 epochs, after iteration 26, no improvement is observed, and overfitting suspected, reduced number of epochs to 25 (early stopping could help)
- With 25 epochs, the best model is at iteration 14, after that, an increase in val loss is observed, the number of epochs is set to 15
- Slight overfitting is observed with 15 epoch, will experiment with scheduler or early stopping (model **struggles in predicting class 2**)
- Added step scheduler where the learning rate is multiplied by 0.1 after every 7 epochs. Best epoch was 6 and there is still sign of overfitting with the minority class (2) is still not performing too well. Will explore **scheduler that changes based on accuracy**.
- **ReduceLROnPlateau** showed a more smooth performance although lower accuracy (0.90 vs 0.89). But, f1 score for class 2 improved slightly and lower overfitting. We will next explore **early stopping** after addressing the imbalances in the dataset regarding class 2.

Exploring focal loss. By adding a scaling factor, focal loss adjusts the learning focus dynamically, especially honing in on the challenging examples that are easily misclassified. The model would focus on minority class and samples where it struggles the most.

<https://medium.com/data-scientists-diary/implementing-focal-loss-in-pytorch-for-class-imbalance-24d8aa3b59d9>

```

# Defining function for focal loss
class FocalLoss(nn.Module):
    def __init__(self, alpha=None, gamma=2, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha # controls class imbalance
        self.gamma = gamma # focuses on hard examples
        self.reduction = reduction

    def forward(self, inputs, targets):
        # Using Cross entropy instead since multi class
        CE_loss = F.cross_entropy(inputs, targets, reduction='none', weight=self.alpha)
        pt = torch.exp(-CE_loss)

        # Apply the focal adjustment
        focal_loss = (1 - pt) ** self.gamma * CE_loss

        # Apply reduction (mean, sum, or no reduction)
        if self.reduction == 'mean':

```

```

        return focal_loss.mean()
    elif self.reduction == 'sum':
        return focal_loss.sum()
    else:
        return focal_loss

# Model Training with Focal loss but without class weights (alpha)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (freezing layers and amend fully connected layer for our labels)
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```

Epoch 1/15
-----
train Loss: 0.1800 Acc: 0.8397
val Loss: 0.1701 Acc: 0.8580

Epoch 2/15
-----
train Loss: 0.1132 Acc: 0.8977
val Loss: 0.1338 Acc: 0.8831

Epoch 3/15
-----
train Loss: 0.0810 Acc: 0.9170
val Loss: 0.1261 Acc: 0.8873

Epoch 4/15
-----
train Loss: 0.0718 Acc: 0.9274
val Loss: 0.1279 Acc: 0.8914

Epoch 5/15
-----
train Loss: 0.0767 Acc: 0.9133
val Loss: 0.1270 Acc: 0.8643

Epoch 6/15
-----
train Loss: 0.0635 Acc: 0.9295
val Loss: 0.1070 Acc: 0.9248

```



```

Epoch 7/15
-----
train Loss: 0.0446 Acc: 0.9441
val Loss: 0.1052 Acc: 0.9207

Epoch 8/15
-----
train Loss: 0.0333 Acc: 0.9577
val Loss: 0.1430 Acc: 0.9102

Epoch 9/15
-----
train Loss: 0.0326 Acc: 0.9661
val Loss: 0.1333 Acc: 0.9186

Epoch 10/15
-----
train Loss: 0.0215 Acc: 0.9786
val Loss: 0.1170 Acc: 0.9228

Epoch 11/15
-----
train Loss: 0.0220 Acc: 0.9770
val Loss: 0.1187 Acc: 0.9269

Epoch 12/15
-----
train Loss: 0.0225 Acc: 0.9719

# Model Training with Focal loss including class weights (alpha)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (freezing layers and amend fully connected layer for our labels)
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

alpha = torch.tensor(class_weights, dtype=torch.float).to(device)
criterion = FocalLoss(alpha=alpha, gamma=2).to(device)
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```
↻ Epoch 1/15
-----
train Loss: 0.5601 Acc: 0.3358
val Loss: 0.5779 Acc: 0.4948

Epoch 2/15
-----
train Loss: 0.3701 Acc: 0.5285
val Loss: 0.5153 Acc: 0.2484

Epoch 3/15
-----
train Loss: 0.3356 Acc: 0.5598
val Loss: 0.5462 Acc: 0.2025

Epoch 4/15
-----
train Loss: 0.2803 Acc: 0.5922
val Loss: 0.4442 Acc: 0.5303

Epoch 5/15
-----
train Loss: 0.1646 Acc: 0.6418
val Loss: 0.4317 Acc: 0.6514

Epoch 6/15
-----
train Loss: 0.1406 Acc: 0.7384
val Loss: 0.5318 Acc: 0.7683

Epoch 7/15
-----
train Loss: 0.1174 Acc: 0.7681
val Loss: 0.5172 Acc: 0.8142

Epoch 8/15
-----
train Loss: 0.1036 Acc: 0.8110
val Loss: 0.4434 Acc: 0.7223

Epoch 9/15
-----
train Loss: 0.1115 Acc: 0.7958
val Loss: 0.5965 Acc: 0.8017

Epoch 10/15
-----
train Loss: 0.1480 Acc: 0.7786
val Loss: 0.6054 Acc: 0.6388

Epoch 11/15
-----
train Loss: 0.0741 Acc: 0.8010
val Loss: 0.4897 Acc: 0.7537

Epoch 12/15
-----
train Loss: 0.0542 Acc: 0.8319
```

When using Focal Loss, we observed a slight improvement in performance. However, unexpectedly, Focal loss without alpha (class weight adjustment) performed better than with alpha.

✎ Using WeightedRandomSampler to investigate if there might be improvement

Instead of class weights like we have done so far, we will add WRS to the Data processing section and look for any improvement, if non is observed, we will use Data Augmentation

```
def data_processingwrs(X_train_raw, y_train_raw):
    # Split into train and validation (80:20)
    X_train, X_val, y_train, y_val = train_test_split(
        X_train_raw, y_train_raw, test_size=0.2, stratify=y_train_raw, random_state=42
    )

    train_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    val_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    train_dataset = maskdataset(X_train, y_train, transform=train_transform)
    val_dataset = maskdataset(X_val, y_val, transform=val_transform)

    # WeightedRandomSampler
    class_counts = np.bincount(y_train)
    class_weights = 1.0 / class_counts
    sample_weights = [class_weights[label] for label in y_train]
    sampler = WeightedRandomSampler(sample_weights, num_samples=len(sample_weights), replacement=True)

    train_loader = DataLoader(train_dataset, batch_size=32, sampler=sampler)
    val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

    return train_loader, val_loader

# Load and preprocess data (with WRS inside this function)
train_loader, val_loader = data_processingwrs(X_train_raw, y_train_raw)

# Load pretrained MobileNetV3
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace the classifier layer (final FC layer) to match 3 output classes
```

```
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train the model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion,
                                       optimizer, scheduler, num_epochs=15)
```



Epoch 1/15

train Loss: 0.2803 Acc: 0.6893

val Loss: 0.4573 Acc: 0.4676

Epoch 2/15

train Loss: 0.1262 Acc: 0.8700

val Loss: 0.2169 Acc: 0.7683

Epoch 3/15

train Loss: 0.1144 Acc: 0.8773

val Loss: 0.1741 Acc: 0.8413

Epoch 4/15

train Loss: 0.0715 Acc: 0.9243

val Loss: 0.1366 Acc: 0.8768

Epoch 5/15

train Loss: 0.0656 Acc: 0.9258

val Loss: 0.1324 Acc: 0.8810

Epoch 6/15

train Loss: 0.0561 Acc: 0.9358

val Loss: 0.1595 Acc: 0.8894

Epoch 7/15

train Loss: 0.0504 Acc: 0.9488

val Loss: 0.1290 Acc: 0.8831

Epoch 8/15

train Loss: 0.0329 Acc: 0.9629

val Loss: 0.1136 Acc: 0.9040

Epoch 9/15

train Loss: 0.0425 Acc: 0.9441

val Loss: 0.1408 Acc: 0.8559

Epoch 10/15

train Loss: 0.0392 Acc: 0.9525

val Loss: 0.1430 Acc: 0.8914

```
Epoch 11/15
-----
train Loss: 0.0266 Acc: 0.9702
val Loss: 0.1309 Acc: 0.8747

Epoch 12/15
-----
train Loss: 0.0193 Acc: 0.9791
```

WRS did not improve performance of the model regarding minority classes

✓ More severe data transformation on minority class 2

We attempted to see if there might be improvement if there a more transformations applied to class 2 but this did not improve the model

```
class maskdatasetaugment(Dataset):
    def __init__(self, images, labels, transform_all=None, transform_class2=None):
        self.images = images
        self.labels = labels
        self.transform_all = transform_all # normal transform for all
        self.transform_class2 = transform_class2 # stronger transform for class 2

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        image = cv2.cvtColor(self.images[idx], cv2.COLOR_BGR2RGB)
        image = Image.fromarray(image)

        label = self.labels[idx]

        # Use special transform for class 2
        if label == 2 and self.transform_class2:
            image = self.transform_class2(image)
        elif self.transform_all:
            image = self.transform_all(image)

        return image, label

def data_processingaug(X_train_raw, y_train_raw):
    # Split training data into train and validation (80:20)
    X_train, X_val, y_train, y_val = train_test_split(X_train_raw, y_train_raw, test_size=0.2,
                                                    stratify=y_train_raw, random_state=42)

    # Base transform for all classes (0 and 1)
    transform_all = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225])
    ])

    # Stronger augmentation for minority class 2
```

```

transform_class2 = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1),
    transforms.RandomRotation(degrees=20),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Validation transform (clean only)
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

# Use the updated custom dataset with per-class augment
train_dataset = maskdatasetaugment(X_train, y_train, transform_all=transform_all, transform_class2=transform_class2)
val_dataset = maskdataset(X_val, y_val, transform=val_transform) # original dataset class is fine here

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

return train_loader, val_loader

# Load and preprocess data
train_loader, val_loader = data_processingaug(X_train_raw, y_train_raw)

# Model set up (freezing layers and amend fully connected layer for our labels)
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)
# scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
# Train model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```
↩ Epoch 1/15
-----
train Loss: 0.5012 Acc: 0.7948
val Loss: 1.7086 Acc: 0.7745

Epoch 2/15
-----
train Loss: 0.2530 Acc: 0.8883
val Loss: 2.2336 Acc: 0.8330

Epoch 3/15
-----
train Loss: 0.2195 Acc: 0.9023
val Loss: 2.5410 Acc: 0.8622

Epoch 4/15
-----
train Loss: 0.1479 Acc: 0.9238
val Loss: 3.4329 Acc: 0.9019

Epoch 5/15
-----
train Loss: 0.1827 Acc: 0.9164
val Loss: 2.0426 Acc: 0.9040

Epoch 6/15
-----
train Loss: 0.1505 Acc: 0.9321
val Loss: 2.4774 Acc: 0.9144

Epoch 7/15
-----
train Loss: 0.1192 Acc: 0.9436
val Loss: 2.3838 Acc: 0.9165

Epoch 8/15
-----
train Loss: 0.0976 Acc: 0.9504
val Loss: 2.6475 Acc: 0.8873

Epoch 9/15
-----
train Loss: 0.1107 Acc: 0.9488
val Loss: 2.6874 Acc: 0.9186

Epoch 10/15
-----
train Loss: 0.1195 Acc: 0.9520
val Loss: 3.2404 Acc: 0.9081

Epoch 11/15
-----
train Loss: 0.1434 Acc: 0.9285
val Loss: 2.0052 Acc: 0.8894

Epoch 12/15
-----
train Loss: 0.1304 Acc: 0.9619
```

```
# Load and preprocess data
```

```
train_loader, val_loader = data_processingaug(X_train_raw, y_train_raw)
```

```
# Model set up (freezing layers and amend fully connected layer for our labels)
model_mobilenetv3 = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv3.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv3.classifier[-1].in_features
model_mobilenetv3.classifier[-1] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = FocalLoss(alpha=torch.tensor(class_weights, dtype=torch.float).to(device), gamma=2)
optimizer = optim.Adam(model_mobilenetv3.parameters(), lr=0.001)
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)
# scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
# Train model
trained_mobilenet = train_mobilenetv3(model_mobilenetv3, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

```
Epoch 1/15
-----
train Loss: 0.1792 Acc: 0.5979
val Loss: 1.7243 Acc: 0.7077

Epoch 2/15
-----
train Loss: 0.1351 Acc: 0.7290
val Loss: 2.2241 Acc: 0.7912

Epoch 3/15
-----
train Loss: 0.0577 Acc: 0.7734
val Loss: 2.0441 Acc: 0.7557

Epoch 4/15
-----
train Loss: 0.0967 Acc: 0.7979
val Loss: 1.6317 Acc: 0.7766

Epoch 5/15
-----
train Loss: 0.0603 Acc: 0.8334
val Loss: 2.1719 Acc: 0.7766

Epoch 6/15
-----
train Loss: 0.0539 Acc: 0.8679
val Loss: 2.1437 Acc: 0.8142

Epoch 7/15
-----
train Loss: 0.0373 Acc: 0.8757
```



```
val Loss: 2.0524 Acc: 0.8330
```

```
Epoch 8/15
```

```
-----
train Loss: 0.0437 Acc: 0.8726
val Loss: 2.0108 Acc: 0.8372
```

```
Epoch 9/15
```

```
-----
train Loss: 0.0231 Acc: 0.8705
val Loss: 2.2103 Acc: 0.8455
```

```
Epoch 10/15
```

```
-----
train Loss: 0.0222 Acc: 0.8883
val Loss: 2.4345 Acc: 0.8309
```

```
Epoch 11/15
```

```
-----
train Loss: 0.0246 Acc: 0.8898
val Loss: 2.1823 Acc: 0.8351
```

```
Epoch 12/15
```

```
-----
train Loss: 0.0224 Acc: 0.8861
```

✚ Exploring other CNN architectures

In this section, we use the backbone of RESNET, VGG Net and MobileNetV2. The model(s) with the highest performance metrics is explored and optimised further. We have not seen significant improvements with MobileNetV3.

```
# RESNET Version
# We believed that RESNET would have a strong foundation for distinguishing subtle differences between masked and unmasked faces with high accuracy.
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (using ResNet50 instead of MobileNetV3)
model_resnet = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_resnet.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_resnet.fc.in_features
model_resnet.fc = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam(model_resnet.parameters(), lr=0.001)
```

```
# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model (using existing training function)
trained_resnet = train_mobilenetv3(model_resnet, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 130MB/s]
Epoch 1/15
-----
train Loss: 0.9910 Acc: 0.6642
val Loss: 0.8887 Acc: 0.8852

Epoch 2/15
-----
train Loss: 0.8168 Acc: 0.7953
val Loss: 0.8189 Acc: 0.8497

Epoch 3/15
-----
train Loss: 0.7151 Acc: 0.8245
val Loss: 0.7915 Acc: 0.7432

Epoch 4/15
-----
train Loss: 0.6443 Acc: 0.8574
val Loss: 0.7395 Acc: 0.8372

Epoch 5/15
-----
train Loss: 0.5950 Acc: 0.8815
val Loss: 0.7368 Acc: 0.8184

Epoch 6/15
-----
train Loss: 0.5957 Acc: 0.8616
val Loss: 0.7240 Acc: 0.8372

Epoch 7/15
-----
train Loss: 0.5878 Acc: 0.8527
val Loss: 0.7362 Acc: 0.8142

Epoch 8/15
-----
train Loss: 0.5806 Acc: 0.8653
val Loss: 0.7355 Acc: 0.8288

Epoch 9/15
-----
train Loss: 0.5674 Acc: 0.8679
val Loss: 0.7270 Acc: 0.8267

Epoch 10/15
-----
train Loss: 0.5822 Acc: 0.8695
val Loss: 0.7293 Acc: 0.8330

Epoch 11/15
-----
train Loss: 0.5747 Acc: 0.8700
```

```
val Loss: 0.7352 Acc: 0.8121
```

```
Epoch 12/15
```

```
# Mobile Net V2 Version
# Previous works showed success with MobileNetV2, we expected good result with either this version or V3.
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (using MobileNetV2)
model_mobilenetv2 = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_mobilenetv2.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layer)
num_features = model_mobilenetv2.classifier[1].in_features
model_mobilenetv2.classifier[1] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam(model_mobilenetv2.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_mobilenetv2 = train_mobilenetv3(model_mobilenetv2, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

 Show hidden output

```
# VGG Version
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
```

```
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model (using existing training function)
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

Downloading: "https://download.pytorch.org/models/vgg16_bn-6c64b313.pth" to /root/.cache/torch/hub/checkpoints/vgg16_bn-6c64b313.pth
100%|██████████| 528M/528M [00:08<00:00, 66.5MB/s]
Epoch 1/15
-----
train Loss: 1.6156 Acc: 0.6924
val Loss: 0.9664 Acc: 0.7056

Epoch 2/15
-----
train Loss: 1.0529 Acc: 0.8428
val Loss: 0.7589 Acc: 0.9165

Epoch 3/15
-----
train Loss: 1.5738 Acc: 0.8590
val Loss: 2.7637 Acc: 0.9144

Epoch 4/15
-----
train Loss: 1.3806 Acc: 0.8930
val Loss: 1.4465 Acc: 0.9019

Epoch 5/15
-----
train Loss: 1.2920 Acc: 0.9238
val Loss: 4.7814 Acc: 0.9269

Epoch 6/15
-----
train Loss: 2.6913 Acc: 0.9138
val Loss: 5.4439 Acc: 0.9061

Epoch 7/15
-----
train Loss: 1.8276 Acc: 0.9154
val Loss: 2.7110 Acc: 0.8873

Epoch 8/15
-----
train Loss: 2.3688 Acc: 0.9070
val Loss: 9.9533 Acc: 0.9123

Epoch 9/15
-----
train Loss: 1.0438 Acc: 0.9619
val Loss: 3.8031 Acc: 0.9353

Epoch 10/15
-----
train Loss: 0.2238 Acc: 0.9661
```

```

val Loss: 4.0808 Acc: 0.9436

Epoch 11/15
-----
train Loss: 0.2217 Acc: 0.9728
val Loss: 5.9008 Acc: 0.9415

Epoch 12/15

```

▼ VGG Net Optimisation

VGG Net resulted in the best performance between the models explored; although we suspect overfitting in epochs 6,7 and 8. Recall (44%) is still not on a desired level for the minority class 2.

```

# VGG Net with Focal Loss
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

# Calculate class weights from distribution
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

alpha = torch.tensor(class_weights, dtype=torch.float).to(device) # Focal loss with alpha (class weights)
criterion = FocalLoss(alpha=alpha, gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

→ Epoch 1/15
-----
train Loss: 1.4957 Acc: 0.5614
val Loss: 0.5463 Acc: 0.5031

Epoch 2/15
-----
train Loss: 1.0473 Acc: 0.7102
val Loss: 0.9799 Acc: 0.8768

Epoch 3/15

```

```
-----  
train Loss: 1.4634 Acc: 0.8386  
val Loss: 2.1039 Acc: 0.9374  
  
Epoch 4/15  
-----  
train Loss: 1.3213 Acc: 0.8444  
val Loss: 1.0818 Acc: 0.8852  
  
Epoch 5/15  
-----  
train Loss: 3.1218 Acc: 0.8031  
val Loss: 1.8026 Acc: 0.7620  
  
Epoch 6/15  
-----  
train Loss: 2.1736 Acc: 0.9154  
val Loss: 3.0209 Acc: 0.9395  
  
Epoch 7/15  
-----  
train Loss: 2.1055 Acc: 0.8778  
val Loss: 13.5732 Acc: 0.8894  
  
Epoch 8/15  
-----  
train Loss: 1.3713 Acc: 0.9112  
val Loss: 3.6766 Acc: 0.9061  
  
Epoch 9/15  
-----  
train Loss: 2.1021 Acc: 0.9164  
val Loss: 12.3114 Acc: 0.9353  
  
Epoch 10/15  
-----  
train Loss: 0.9829 Acc: 0.9614  
val Loss: 10.8563 Acc: 0.9374  
  
Epoch 11/15  
-----  
train Loss: 1.7650 Acc: 0.9556  
val Loss: 9.8028 Acc: 0.9353  
  
Epoch 12/15  
-----  
train Loss: 1.6755 Acc: 0.9457  
  
# VGG Net with Focal Loss (no alpha)  
# Load and preprocess data  
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)  
  
# Model set up (using VGG16 with batch normalization)  
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)  
  
# Freeze feature extraction layers  
for param in model_vgg.features.parameters():  
    param.requires_grad = False  
  
# Replace fully connected layer (last layers)
```

```
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```



Epoch 1/15

```
-----
train Loss: 0.2718 Acc: 0.8360
val Loss: 0.0876 Acc: 0.9186
```

Epoch 2/15

```
-----
train Loss: 0.1234 Acc: 0.9227
val Loss: 0.0991 Acc: 0.9290
```

Epoch 3/15

```
-----
train Loss: 0.0782 Acc: 0.9389
val Loss: 0.2759 Acc: 0.9040
```

Epoch 4/15

```
-----
train Loss: 0.0934 Acc: 0.9410
val Loss: 0.1108 Acc: 0.9395
```

Epoch 5/15

```
-----
train Loss: 0.1004 Acc: 0.9582
val Loss: 0.4336 Acc: 0.9102
```

Epoch 6/15

```
-----
train Loss: 0.1057 Acc: 0.9655
val Loss: 0.2428 Acc: 0.9102
```

Epoch 7/15

```
-----
train Loss: 0.0762 Acc: 0.9697
val Loss: 0.5309 Acc: 0.9332
```

Epoch 8/15

```
-----
train Loss: 0.0647 Acc: 0.9760
val Loss: 0.2067 Acc: 0.9395
```

Epoch 9/15

```
-----
train Loss: 0.0108 Acc: 0.9916
val Loss: 0.1994 Acc: 0.9395
```

Epoch 10/15

```
-----
train Loss: 0.0122 Acc: 0.9932
val Loss: 0.1874 Acc: 0.9395
```

```
Epoch 11/15
-----
train Loss: 0.0106 Acc: 0.9911
val Loss: 0.1941 Acc: 0.9395
```

```
Epoch 12/15
-----
train Loss: 0.0111 Acc: 0.9948
```

Focal loss without alpha (i.e not adjusted for class weights) led to higher precision and f1-score while recall stayed the same at 44%. Focal Loss also performed better than Cross-entropy

Weighted Random Sampler is also explored again to address the imbalances although this did not improve performance for MobileNetV3.

```
# VGG Version with Weighted Random Sampler (Focal loss)
# Load and preprocess data
train_loader, val_loader = data_processingwrs(X_train_raw, y_train_raw)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model (using your existing training function)
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

```
↩ Epoch 1/15
-----
train Loss: 0.3766 Acc: 0.7044
val Loss: 0.1633 Acc: 0.8539

Epoch 2/15
-----
train Loss: 0.0777 Acc: 0.9321
val Loss: 0.0803 Acc: 0.9415

Epoch 3/15
-----
train Loss: 0.0618 Acc: 0.9603
val Loss: 0.1426 Acc: 0.9457

Epoch 4/15
-----
```



```
train Loss: 0.1446 Acc: 0.9452
val Loss: 0.3974 Acc: 0.9332
```

```
Epoch 5/15
```

```
-----
train Loss: 0.1876 Acc: 0.9629
val Loss: 0.8279 Acc: 0.8873
```

```
Epoch 6/15
```

```
-----
train Loss: 0.1595 Acc: 0.9624
val Loss: 0.7553 Acc: 0.8664
```

```
Epoch 7/15
```

```
-----
train Loss: 0.0989 Acc: 0.9786
val Loss: 0.3419 Acc: 0.9457
```

```
Epoch 8/15
```

```
-----
train Loss: 0.0289 Acc: 0.9937
val Loss: 0.2876 Acc: 0.9457
```

```
Epoch 9/15
```

```
-----
train Loss: 0.0197 Acc: 0.9885
val Loss: 0.2932 Acc: 0.9478
```

```
Epoch 10/15
```

```
-----
train Loss: 0.0183 Acc: 0.9932
val Loss: 0.3172 Acc: 0.9478
```

```
Epoch 11/15
```

```
-----
train Loss: 0.0253 Acc: 0.9937
val Loss: 0.3068 Acc: 0.9457
```

```
Epoch 12/15
```

```
-----
train Loss: 0.0153 Acc: 0.9940
```

Weighted Random Sampler did not improve the prediction of class 2

As previously observed, the distribution of the data is Class 0: 376 samples; Class 1: 1,940 samples; Class 2: 78 samples. Class 2 is severely underrepresented, hence we will oversample class 2 in an attempt to help the model

```
X_class2 = [x for x, y in zip(X_train_raw, y_train_raw) if y == 2]
y_class2 = [2] * len(X_class2)
```

```
len(y_class2)
```

```
↔ 78
```

```
# Dataset preparation/wrapping
# To prepare the dataset to be used with PyTorch's Dataloader and implement
#transformations for training and validation
```

```

class maskdataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels
        self.transform = transform

    def __len__(self): # define the number of samples in the dataset
        return len(self.images)

    def __getitem__(self, idx):
        # Convert image to uint8 to handle unexpected float type
        image = cv2.cvtColor(self.images[idx].astype(np.uint8), cv2.COLOR_BGR2RGB) #cv2 uses BGR
        image = Image.fromarray(image)

        if self.transform:
            image = self.transform(image)

        return image, self.labels[idx]

```

✓ Data Augmentation (Oversampling Class 2)

We have explored different losses and WeightedRaandomSampler along with assigning different weights to the classes. Recall is still underperforming and in the next part, we attempt to improve performance of the model to predict Class 2 by oversampling

```

# Import necessary libraries
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation settings (rotation, horizontal and vertical shifts, changes in brightness)
# This would determine how the images are randomly transformed
datagen = ImageDataGenerator(rotation_range=20, width_shift_range=0.2, height_shift_range=0.2,
                             brightness_range=[0.7, 1.3], shear_range=0.15, fill_mode='reflect')

# Extracting Class 2 (improperly worn mask) images from the dataset
X_minority_2 = [img for img, label in zip(X_train_raw, y_train_raw) if label == 2]
y_minority_2 = [2] * len(X_minority_2)

# Function to generate augmented samples for each original samples
def generate_augmented_samples(images, labels, n_samples):
    augmented_images = []
    augmented_labels = []

    for img, label in zip(images, labels):
        img_array = np.expand_dims(img, axis=0) # Add batch dimension

        # Generate augmented versions of a particular image
        aug_iter = datagen.flow(img_array, np.array([label]),
                                batch_size=1, shuffle=False)

        for _ in range(n_samples):
            aug_img = next(aug_iter)[0][0] # Get the augmented image
            augmented_images.append(aug_img)
            augmented_labels.append(label)

    return augmented_images, augmented_labels

```

```

n_samples_per_img = 1 # Class 2 get 1 additional image (doubling the samples of class 2)

aug_images_2, aug_labels_2 = generate_augmented_samples(X_minority_2, y_minority_2, n_samples_per_img)

# Combining aug data with original data
if isinstance(X_train_raw, list):
    X_train_raw_aug = X_train_raw + aug_images_2
    y_train_raw_aug = list(y_train_raw) + aug_labels_2
# Converting to lists since np arrays of different sizes cannot be combined together
# As a list, the new iamges are appended to the dataset
else:
    X_train_raw_list = [X_train_raw[i] for i in range(len(X_train_raw))] # was X_train_sift
    X_train_raw_aug = X_train_raw_list + aug_images_2
    y_train_raw_aug = list(y_train_raw) + aug_labels_2

# VGG Net with Focal Loss (no alpha)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw_aug, y_train_raw_aug)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```

🔄 Epoch 1/15
-----
train Loss: 0.3529 Acc: 0.7830
val Loss: 0.1072 Acc: 0.8929

Epoch 2/15
-----
train Loss: 0.1100 Acc: 0.9150
val Loss: 0.1620 Acc: 0.8929

Epoch 3/15
-----
train Loss: 0.1291 Acc: 0.9262
val Loss: 0.1866 Acc: 0.9111

Epoch 4/15
-----
train Loss: 0.1461 Acc: 0.9287

```

```

val Loss: 0.1198 Acc: 0.9131

Epoch 5/15
-----
train Loss: 0.0698 Acc: 0.9555
val Loss: 0.1133 Acc: 0.9051

Epoch 6/15
-----
train Loss: 0.0545 Acc: 0.9732
val Loss: 0.1687 Acc: 0.9273

Epoch 7/15
-----
train Loss: 0.1252 Acc: 0.9555
val Loss: 0.2031 Acc: 0.9273

Epoch 8/15
-----
train Loss: 0.1308 Acc: 0.9621
val Loss: 1.1829 Acc: 0.8586

Epoch 9/15
-----
train Loss: 0.2018 Acc: 0.9514
val Loss: 0.1798 Acc: 0.9354

Epoch 10/15
-----
train Loss: 0.0708 Acc: 0.9737
val Loss: 0.2554 Acc: 0.9293

Epoch 11/15
-----
train Loss: 0.0424 Acc: 0.9808
val Loss: 0.4420 Acc: 0.9010

Epoch 12/15
-----
train Loss: 0.0071 Acc: 0.9742

```

By doubling the sample of class 2, we observe overfitting but a significant boost in **recall** is seen. We will explore some ways to mitigate overfitting such as early stopping or L1/L2 regularization or a combination of both or dropout

```

# VGG Net with Focal Loss (no alpha) + Weight Decay(0.0001) (L2 norm)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw_aug, y_train_raw_aug)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

```

```
criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001, weight_decay=0.0001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

```
Epoch 1/15
-----
train Loss: 0.3689 Acc: 0.8113
val Loss: 0.1309 Acc: 0.8788

Epoch 2/15
-----
train Loss: 0.1120 Acc: 0.9125
val Loss: 0.1541 Acc: 0.9010

Epoch 3/15
-----
train Loss: 0.1047 Acc: 0.9267
val Loss: 0.1326 Acc: 0.9131

Epoch 4/15
-----
train Loss: 0.0766 Acc: 0.9423
val Loss: 0.1396 Acc: 0.9071

Epoch 5/15
-----
train Loss: 0.0680 Acc: 0.9570
val Loss: 0.1639 Acc: 0.9131

Epoch 6/15
-----
train Loss: 0.0963 Acc: 0.9459
val Loss: 0.2210 Acc: 0.9030

Epoch 7/15
-----
train Loss: 0.0282 Acc: 0.9798
val Loss: 0.0983 Acc: 0.9354

Epoch 8/15
-----
train Loss: 0.0161 Acc: 0.9858
val Loss: 0.1090 Acc: 0.9354

Epoch 9/15
-----
train Loss: 0.0109 Acc: 0.9914
val Loss: 0.1248 Acc: 0.9313

Epoch 10/15
-----
train Loss: 0.0080 Acc: 0.9919
val Loss: 0.1010 Acc: 0.9374

Epoch 11/15
-----
```

```
train Loss: 0.0112 Acc: 0.9904
val Loss: 0.0998 Acc: 0.9354

Epoch 12/15
-----
train Loss: 0.0062 Acc: 0.9924

# VGG Net with Focal Loss (no alpha) + Weight Decay (0.001) (L2 norm)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw_aug, y_train_raw_aug)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001, weight_decay=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

↩ Epoch 1/15
-----
train Loss: 0.4221 Acc: 0.7815
val Loss: 0.1466 Acc: 0.8848

Epoch 2/15
-----
train Loss: 0.1261 Acc: 0.9014
val Loss: 0.1485 Acc: 0.9071

Epoch 3/15
-----
train Loss: 0.1181 Acc: 0.8993
val Loss: 0.1095 Acc: 0.9051

Epoch 4/15
-----
train Loss: 0.1024 Acc: 0.9277
val Loss: 0.0784 Acc: 0.9273

Epoch 5/15
-----
train Loss: 0.0802 Acc: 0.9368
val Loss: 0.0840 Acc: 0.9293

Epoch 6/15
-----
train Loss: 0.0765 Acc: 0.9363
```

```

val Loss: 0.0847 Acc: 0.9232

Epoch 7/15
-----
train Loss: 0.0612 Acc: 0.9474
val Loss: 0.1259 Acc: 0.9192

Epoch 8/15
-----
train Loss: 0.0829 Acc: 0.9383
val Loss: 0.1039 Acc: 0.9212

Epoch 9/15
-----
train Loss: 0.0321 Acc: 0.9712
val Loss: 0.0847 Acc: 0.9313

Epoch 10/15
-----
train Loss: 0.0185 Acc: 0.9828
val Loss: 0.0856 Acc: 0.9333

Epoch 11/15
-----
train Loss: 0.0127 Acc: 0.9889
val Loss: 0.0919 Acc: 0.9293

Epoch 12/15
-----
train Loss: 0.0115 Acc: 0.9914

# VGG Net (with drop out in the classifier) (Turning off 50% of neurons for two layers)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace the classifier with a new one that includes dropout
model_vgg.classifier = nn.Sequential(
    nn.Linear(512 * 7 * 7, 4096),
    nn.ReLU(True),
    nn.Dropout(p=0.5), # First dropout layer
    nn.Linear(4096, 4096),
    nn.ReLU(True),
    nn.Dropout(p=0.5), # Second dropout layer
    nn.Linear(4096, 3) # Output layer with 3 classes
)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001, weight_decay=0.001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```
↻ Epoch 1/15
-----
train Loss: 0.3456 Acc: 0.8513
val Loss: 0.1096 Acc: 0.8990

Epoch 2/15
-----
train Loss: 0.1033 Acc: 0.9216
val Loss: 0.1978 Acc: 0.9071

Epoch 3/15
-----
train Loss: 0.0828 Acc: 0.9327
val Loss: 0.1031 Acc: 0.9192

Epoch 4/15
-----
train Loss: 0.0647 Acc: 0.9514
val Loss: 0.1100 Acc: 0.9111

Epoch 5/15
-----
train Loss: 0.0580 Acc: 0.9479
val Loss: 0.1055 Acc: 0.9172

Epoch 6/15
-----
train Loss: 0.0594 Acc: 0.9545
val Loss: 0.1012 Acc: 0.9273

Epoch 7/15
-----
train Loss: 0.0496 Acc: 0.9590
val Loss: 0.1402 Acc: 0.9010

Epoch 8/15
-----
train Loss: 0.0523 Acc: 0.9575
val Loss: 0.1650 Acc: 0.9051

Epoch 9/15
-----
train Loss: 0.0421 Acc: 0.9646
val Loss: 0.0928 Acc: 0.9172

Epoch 10/15
-----
train Loss: 0.0296 Acc: 0.9727
val Loss: 0.1022 Acc: 0.9172

Epoch 11/15
-----
train Loss: 0.0182 Acc: 0.9833
val Loss: 0.0828 Acc: 0.9333

Epoch 12/15
-----
train Loss: 0.0112 Acc: 0.9889
```

Higher dropout probability hurt performance (we are aiming for a recall of at least above 50% for underrepreseted class)

Below we explore early stopping as a way to mitigate overfitting

```
# Main training function adjusted to include early stopping
# Early stopping is triggered based on val loss instead of accuracy to prevent overfitting
def train_function_es(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15):
    model = model.to(device)
    # Track best model
    best_model_wts = copy.deepcopy(model.state_dict())
    patience = 5 # for Early stopping (number of epochs to wait)
    counter = 0
    best_loss = float('inf')
    early_stop = False

    since = time.time() # Time tracking
    # Performance metrics
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_acc': [],
        'best_epoch': 0
    }
    # Main training loop going over specified number of epochs
    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")
        print('-' * 20)
        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            model.train() if phase == 'train' else model.eval()
            dataloader = train_loader if phase == 'train' else val_loader

            running_loss = 0.0
            running_corrects = 0
            all_preds = []
            all_labels = []
            # Iterate over data
            for inputs, labels in dataloader:
                inputs, labels = inputs.to(device), labels.to(device)
                # Zero the parameter gradients
                optimizer.zero_grad()
                # Forward pass
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)
                # Backward + optimize only if in training phase
                if phase == 'train':
                    loss.backward()
                    optimizer.step()

            # Statistics
            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)
            # Collect predictions and labels for metrics
            if phase == 'val':
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(labels.cpu().numpy())
        # Calculate epoch statistics after each epoch
```

```

dataset_size = len(dataloader.dataset)
epoch_loss = running_loss / dataset_size
epoch_acc = running_corrects.double() / dataset_size

print(f"{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}")
# Deep copy the model with best accuracy on validation set
if phase == 'val':
    history['val_loss'].append(epoch_loss)
    history['val_acc'].append(epoch_acc.item())
    # Print detailed metrics at the last epoch or if improvements
    if epoch_loss < best_loss:
        print(f"Validation loss improved from {best_loss:.4f} to {epoch_loss:.4f}")
        best_loss = epoch_loss
        best_model_wts = copy.deepcopy(model.state_dict())
        history['best_epoch'] = epoch
        counter = 0
    else:
        counter += 1
        print(f"No improvement in val loss for {counter} epoch(s).")

        if counter >= patience:
            print("Early stopping triggered.")
            early_stop = True

    if epoch == num_epochs - 1 or counter == 0:
        print(classification_report(all_labels, all_preds, target_names=['0', '1', '2']))

else:
    history['train_loss'].append(epoch_loss)
# Scheduler (based on val accuracy)
if scheduler is not None and phase == 'val':
    scheduler.step(epoch_acc)

if early_stop:
    print(f"Stopping early at epoch {epoch+1}")
    break
print()
# Training complete
time_elapsed = time.time() - since
print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")
print(f"Best val loss: {best_loss:.4f} (epoch {history['best_epoch'] + 1})")
# Load best model weights
model.load_state_dict(best_model_wts)

model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

```

```
print("Classification Report (Best Model):")
print(classification_report(all_labels, all_preds, target_names=['0', '1', '2']))
return model, history

# VGG Net with Focal Loss (no alpha) + Weight Decay (L2 norm)
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw_aug, y_train_raw_aug)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

criterion = FocalLoss(gamma=2).to(device)
optimizer = optim.Adam(model_vgg.parameters(), lr=0.001, weight_decay=0.0001)

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_function_es(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)
```

↩ Epoch 1/15

train Loss: 0.3360 Acc: 0.7962
val Loss: 0.1957 Acc: 0.8444
Validation loss improved from inf to 0.1957

	precision	recall	f1-score	support
0	1.00	0.31	0.47	75
1	0.86	0.99	0.92	389
2	0.43	0.29	0.35	31
accuracy			0.84	495
macro avg	0.76	0.53	0.58	495
weighted avg	0.85	0.84	0.82	495

Epoch 2/15

train Loss: 0.1080 Acc: 0.9054
val Loss: 0.1329 Acc: 0.9111
Validation loss improved from 0.1957 to 0.1329

	precision	recall	f1-score	support
0	0.85	0.81	0.83	75
1	0.92	0.99	0.95	389
2	1.00	0.16	0.28	31
accuracy			0.91	495
macro avg	0.92	0.65	0.69	495
weighted avg	0.91	0.91	0.89	495



```
Epoch 3/15
-----
train Loss: 0.0891 Acc: 0.9408
val Loss: 0.1213 Acc: 0.8949
Validation loss improved from 0.1329 to 0.1213
      precision    recall  f1-score   support

         0         0.77      0.91      0.83         75
         1         0.98      0.91      0.94        389
         2         0.47      0.68      0.55         31

   accuracy
macro avg      0.74      0.83      0.78        495
weighted avg      0.91      0.89      0.90        495
```

```
Epoch 4/15
-----
train Loss: 0.0906 Acc: 0.9393
val Loss: 0.2192 Acc: 0.9253
No improvement in val loss for 1 epoch(s).
```

```
Epoch 5/15
-----
train Loss: 0.1591 Acc: 0.9332
val Loss: 0.2744 Acc: 0.9020
```

Early stopping is found to hurt performance when implemented not achieving above 90% accuracy compared to the other methods implemented.

The best model with adjustment to mitigate overfitting while still maintaining relatively good recall is the VGG model with Focal Loss and Weight Decay of 0.001 and dropout in FCL of 0.5 probability. We will attempt to fine tune this model in the next section.

✓ Fine Tuning (Unfreezing last conv layer)

BEST MODEL FROM CNN (Transfer learning: VGG Net)

```
# VGG16 (Fine tuning model with: 0.5 Dropout, Weight Decay 0.001, Focal Loss without alpha)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# First freeze all feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

# Unfreeze only the last convolutional block (layers 33 onwards)
for param in model_vgg.features[33:].parameters():
    param.requires_grad = True

# Replace the classifier with a new one that includes dropout
model_vgg.classifier = nn.Sequential(
    nn.Linear(512 * 7 * 7, 4096),
    nn.ReLU(True),
```

```

nn.Dropout(p=0.5), # First dropout layer
nn.Linear(4096, 4096),
nn.ReLU(True),
nn.Dropout(p=0.5), # Second dropout layer
nn.Linear(4096, 3) # Output layer with 3 classes
)

criterion = FocalLoss(gamma=2).to(device) # Loss

# Create optimizer with different learning rates
optimizer = optim.Adam([
    {'params': model_vgg.features[33:].parameters(), 'lr': 0.0001}, # Lower learning rate for fine-tuning
    {'params': model_vgg.classifier.parameters(), 'lr': 0.001}      # Higher learning rate for classifier
], weight_decay=0.001) # Weight Decay

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```

Epoch 1/15
-----
train Loss: 0.3236 Acc: 0.8589
val Loss: 0.0993 Acc: 0.9152

```

```

Epoch 2/15
-----
train Loss: 0.0459 Acc: 0.9616
val Loss: 0.0667 Acc: 0.9374

```

```

Epoch 3/15
-----
train Loss: 0.0188 Acc: 0.9813
val Loss: 0.0667 Acc: 0.9394

```

```

Epoch 4/15
-----
train Loss: 0.0206 Acc: 0.9823
val Loss: 0.0774 Acc: 0.9333

```

```

Epoch 5/15
-----
train Loss: 0.0198 Acc: 0.9828
val Loss: 0.0801 Acc: 0.9414

```

```

Epoch 6/15
-----
train Loss: 0.0065 Acc: 0.9939
val Loss: 0.0700 Acc: 0.9495

```

```

Epoch 7/15
-----
train Loss: 0.0071 Acc: 0.9949
val Loss: 0.0690 Acc: 0.9475

```

```

Epoch 8/15
-----
train Loss: 0.0143 Acc: 0.9868
val Loss: 0.0748 Acc: 0.9556

```

```

Epoch 9/15
-----
train Loss: 0.0030 Acc: 0.9975
val Loss: 0.0725 Acc: 0.9636

Epoch 10/15
-----
train Loss: 0.0095 Acc: 0.9894
val Loss: 0.0889 Acc: 0.9515

Epoch 11/15
-----
train Loss: 0.0043 Acc: 0.9939
val Loss: 0.1022 Acc: 0.9414

Epoch 12/15
-----
train Loss: 0.0096 Acc: 0.9929

# VGG Net with Cross Entropy (FINE TUNED (Last Conv Layer not frozen))
# We attempted fine tuning with cross entropy to see if there might be a misopportunity.
# Load and preprocess data
train_loader, val_loader = data_processing(X_train_raw, y_train_raw)

# Model set up (using VGG16 with batch normalization)
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Freeze feature extraction layers
for param in model_vgg.features.parameters():
    param.requires_grad = False

for layer in model_vgg.features[33:]:
    for param in layer.parameters():
        param.requires_grad = True

# Replace fully connected layer (last layers)
num_features = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_features, 3)

# Calculate class weights
y_train_np = np.array(y_train_raw)
class_counts = np.bincount(y_train_np)
total_samples = len(y_train_np)
class_weights = total_samples / (len(class_counts) * class_counts)

criterion = nn.CrossEntropyLoss(weight=torch.tensor(class_weights, dtype=torch.float).to(device))
optimizer = optim.Adam([
    {'params': [p for p in model_vgg.features.parameters() if p.requires_grad], 'lr': 0.0001},
    {'params': model_vgg.classifier.parameters(), 'lr': 0.001}
])

# Adding a scheduler that would decrease based on accuracy
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.1, patience=2)

# Train model
trained_vgg = train_mobilenetv3(model_vgg, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15)

```

```
↩ Epoch 1/15
-----
train Loss: 1.2140 Acc: 0.7535
val Loss: 0.6936 Acc: 0.8977

Epoch 2/15
-----
train Loss: 0.3925 Acc: 0.9269
val Loss: 0.5473 Acc: 0.9332

Epoch 3/15
-----
train Loss: 0.2945 Acc: 0.9373
val Loss: 0.5071 Acc: 0.9478

Epoch 4/15
-----
train Loss: 0.1820 Acc: 0.9708
val Loss: 0.8261 Acc: 0.9457

Epoch 5/15
-----
train Loss: 0.2308 Acc: 0.9634
val Loss: 0.4352 Acc: 0.9395

Epoch 6/15
-----
train Loss: 0.3278 Acc: 0.9645
val Loss: 2.2284 Acc: 0.8559

Epoch 7/15
-----
train Loss: 0.2012 Acc: 0.9540
val Loss: 0.9179 Acc: 0.9353

Epoch 8/15
-----
train Loss: 0.0655 Acc: 0.9744
val Loss: 0.8249 Acc: 0.9353

Epoch 9/15
-----
train Loss: 0.0461 Acc: 0.9807
val Loss: 0.8512 Acc: 0.9478

Epoch 10/15
-----
train Loss: 0.0438 Acc: 0.9896
val Loss: 0.8525 Acc: 0.9478

Epoch 11/15
-----
train Loss: 0.0258 Acc: 0.9922
val Loss: 0.8257 Acc: 0.9436

Epoch 12/15
-----
train Loss: 0.0370 Acc: 0.9880
```

Overfitting is definitely happening despite L2 Norm and Dropout. We will use the weight from epoch 9 to test our model on the test set

✓ Testing VGG Net

```
# Same transform as used for validation
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

# Create test dataset and dataloader
test_dataset = maskdataset(X_test_raw, y_test_raw, transform=test_transform)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# Evaluation mode
model_vgg.eval()
model_vgg.to(device)

# Collect predictions
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model_vgg(inputs)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Evaluate
from sklearn.metrics import classification_report, accuracy_score

print("Test Accuracy:", accuracy_score(all_labels, all_preds))
print("Classification Report:")
print(classification_report(all_labels, all_preds))
```

 Test Accuracy: 0.9344978165938864
 Classification Report:

	precision	recall	f1-score	support
0	0.94	0.86	0.90	51
1	0.98	0.96	0.97	388
2	0.37	0.58	0.45	19
accuracy			0.93	458
macro avg	0.76	0.80	0.77	458
weighted avg	0.95	0.93	0.94	458

```
# Save the best model (weights only)
# torch.save(trained_vgg[0].state_dict(), "/content/drive/My Drive/CV CW/vgg_finetuned_mask_classifier.pth")
```


Loading the weights for qualitative comparison of the models

```
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
import numpy as np
import cv2
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, accuracy_score

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

# Replace the classifier with the same architecture used during training
model_vgg.classifier = torch.nn.Sequential(
    torch.nn.Linear(512 * 7 * 7, 4096),
    torch.nn.ReLU(True),
    torch.nn.Dropout(p=0.5), # First dropout layer
    torch.nn.Linear(4096, 4096),
    torch.nn.ReLU(True),
    torch.nn.Dropout(p=0.5), # Second dropout layer
    torch.nn.Linear(4096, 3) # Output layer with 3 classes
)

# Load the saved weights
model_vgg.load_state_dict(torch.load("/content/drive/MyDrive/CV CW/vgg_finetuned_mask_classifier.pth"))
model_vgg.to(device)
model_vgg.eval()
```



Show hidden output

```
# Define the test transforms (same as used during testing)
test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Custom Dataset class for the test set
# (Assuming X_test_raw and y_test are the raw test images and labels from your earlier code)
class CustomDataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images # List of raw images (loaded via cv2.imread, in BGR format)
        self.labels = labels # List or array of labels
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
```

```

img = self.images[idx]
label = self.labels[idx]
# Convert BGR to RGB (since cv2.imread loads in BGR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Convert to PIL Image for torchvision transformations
from PIL import Image
img_pil = Image.fromarray(img_rgb)
# Apply transforms
if self.transform:
    img_pil = self.transform(img_pil)
return img_pil, label

# Create the test dataset and dataloader
test_dataset = CustomDataset(X_test_raw, y_test_raw, transform=test_transforms)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

all_preds = []
all_labels = []
all_images = [] # To store the original RGB images for visualization

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model_vgg(inputs)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        # Store the original images (before transforms) for visualization
        for i in range(inputs.size(0)):
            img = X_test_raw[len(all_images)] # Get the corresponding raw image
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            all_images.append(img_rgb)

# Evaluation
print("Test Accuracy:", accuracy_score(all_labels, all_preds))
print("Classification Report:")
print(classification_report(all_labels, all_preds))

↗ Test Accuracy: 0.9344978165938864
Classification Report:

```

	precision	recall	f1-score	support
0	0.94	0.86	0.90	51
1	0.98	0.96	0.97	388
2	0.37	0.58	0.45	19
accuracy			0.93	458
macro avg	0.76	0.80	0.77	458
weighted avg	0.95	0.93	0.94	458

```

# Visualisation of first 10 images (similar to other models)
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()

for i in range(10):

```

```
img_rgb = all_images[i] # Already in RGB format
ax[i].imshow(img_rgb)
ax[i].set_title(f"Label: {all_labels[i]}\nPred: {all_preds[i]}")
ax[i].set_axis_off()
```

```
plt.tight_layout()
plt.show()
```



```
# Randomly generated 40 images with their labels and predictions
fig, axes = plt.subplots(4, 10, figsize=(20, 10), sharex=True, sharey=True)
ax = axes.ravel()
```

```
# Generate 40 random indices
random_indices = np.random.randint(0, len(all_images), 40)
```

```
for i in range(40):
    img_rgb = all_images[random_indices[i]] # Use random indices
    ax[i].imshow(img_rgb)
    ax[i].set_title(f"Label: {all_labels[random_indices[i]]}\nPred: {all_preds[random_indices[i]]}")
    ax[i].set_axis_off()
```

```
plt.tight_layout()
plt.show()
```



```
# Preprocess test images
X_test_processed = preprocess_hog(X_test_raw, image_size=(128, 128))

# Extract HOG features
X_test_hog = hog_features(X_test_processed)

# Scale the features using the same scaler that was fit on training data
X_test_hog_scaled = scaler.transform(X_test_hog)

# Make predictions
y_test_pred = best_model.predict(X_test_hog_scaled)
```

```
# Evaluate results
test_accuracy = accuracy_score(y_test_raw, y_test_pred)
print("Test Classification Report:")
print(classification_report(y_test_raw, y_test_pred))
```

```
↗ Test Classification Report:
              precision    recall  f1-score   support

     0         0.24        0.67        0.36         51
     1         0.92        0.73        0.81        388
     2         0.42        0.26        0.32         19

 accuracy          0.70        0.70        0.70        458
 macro avg         0.53        0.55        0.50        458
 weighted avg      0.83        0.70        0.74        458
```

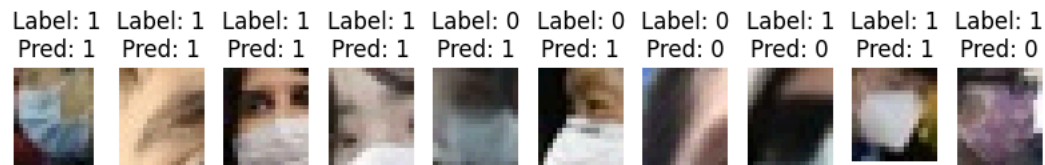
```
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
for i in range(10):
    # Convert BGR to RGB
    img_rgb = cv2.cvtColor(X_test_raw[i], cv2.COLOR_BGR2RGB)
    ax[i].imshow(img_rgb)
    ax[i].set_title(f"Label: {y_test_raw[i]}\nPred: {y_test_pred[i]}")
    ax[i].set_axis_off()
```

```
plt.show()
```



```
# Randomly generated images
fig, axes = plt.subplots(2, 10, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
for i in range(20):
    idx = np.random.randint(0, len(X_test_raw))
    img = X_test_raw[idx]
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    ax[i].imshow(img_rgb)
```

```
ax[i].set_title(f"Label: {y_test_raw[i]}\nPred: {y_test_pred[i]}")
ax[i].set_axis_off()
plt.tight_layout()
plt.show()
```



✓ Testing SIFT+SVM

```
# Initialize lists to store SIFT features and labels for the test set
des_list_test = []
y_test_sift = []

# Extract SIFT features from test images
for i in range(len(X_test_raw)):
    # Convert image to grayscale (SIFT works on grayscale images)
    img = cv2.cvtColor(X_test_raw[i].astype(np.uint8), cv2.COLOR_RGB2GRAY)

    # Detect keypoints and compute SIFT descriptors
    kp, des = sift.detectAndCompute(img, None)

    # Append descriptors if they exist
    if des is not None:
        des_list_test.append(des)
        y_test_sift.append(y_test_raw[i])
    else:
        print(f"No descriptors for test image {i}")

# Convert descriptors to histograms using the trained MiniBatchKMeans
hist_list_test = []
for des in des_list_test:
    hist = np.zeros(k) # k is the number of clusters (from MiniBatchKMeans)
    idx = kmeans.predict(des)
    for j in idx:
        hist[j] = hist[j] + (1 / len(des))
    hist_list_test.append(hist)

# Convert to array for prediction
```

```
des_array_test = np.vstack(hist_list_test)

# Using the trained classifier (best from SIFT+SVM)
y_pred_test = classifier_aug.predict(des_array_test)

print("\nClassification report for test set:")
print(classification_report(y_test_sift, y_pred_test))

# Visualize some predictions
fig, axes = plt.subplots(2, 5, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
for i in range(10):
    img_rgb = cv2.cvtColor(X_test_raw[i], cv2.COLOR_BGR2RGB)
    ax[i].imshow(img_rgb)
    ax[i].set_title(f"Label: {y_test_raw[i]}\nPred: {y_pred_test[i]}")
    ax[i].set_axis_off()
plt.tight_layout()
plt.show()
```

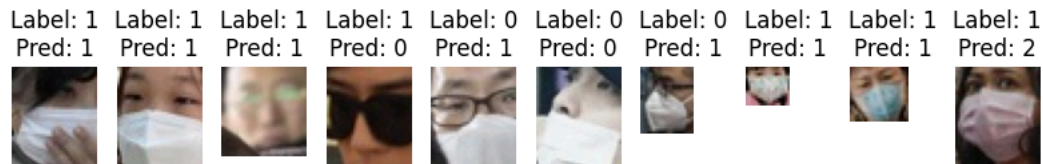
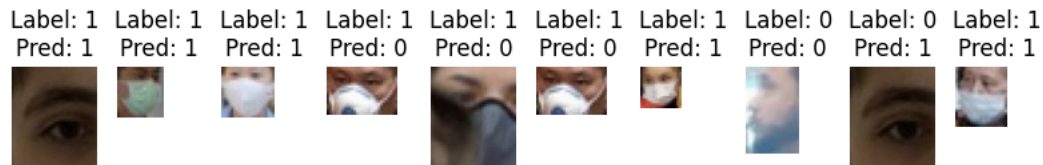
No descriptors for test image 7
No descriptors for test image 68
No descriptors for test image 175
No descriptors for test image 453

Classification report for test set:

	precision	recall	f1-score	support
0	0.32	0.62	0.42	50
1	0.94	0.70	0.80	385
2	0.12	0.47	0.20	19
accuracy			0.68	454
macro avg	0.46	0.60	0.47	454
weighted avg	0.84	0.68	0.73	454



```
# Randomly generated images
fig, axes = plt.subplots(2, 10, figsize=(8, 4), sharex=True, sharey=True)
ax = axes.ravel()
for i in range(20):
    idx = np.random.randint(0, len(X_test_raw))
    img = X_test_raw[idx]
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    ax[i].imshow(img_rgb)
    ax[i].set_title(f"Label: {y_test_raw[i]}\nPred: {y_pred_test[i]}")
    ax[i].set_axis_off()
plt.tight_layout()
plt.show()
```



There is not sign of overfitting since accuracy of 66% is very close to training accuracy of 67%. The model struggles with minority classes and is not confident in predicting classes 0 and 2 with precision being 0.28 and 0.08 respectively. From the visualisation as well, we can see that the model predicts most of class 1 very well but would sometime confuse class 1 (mask) with class 0 (no mask). This illustrates the limitations of SIFT in this case. SIFT failed to identify features in some images and also led to the misclassification as observed above.

Video Showcase

In this section we test our best model (VGG Net) on a video

We will use CV2 Cascade Classifier to detect face in the video and add bounding boxes

```
# Import necessary libraries
import torch
import torchvision.models as models
import torchvision.transforms as transforms
import numpy as np
import cv2
import matplotlib.pyplot as plt
from PIL import Image
import matplotlib.animation as animation
from IPython.display import HTML
```



```

# Define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Load the Haar Cascade Classifier for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')

# Load the VGG16 model
model_vgg = models.vgg16_bn(weights=models.VGG16_BN_Weights.DEFAULT)

📄 Downloading: "https://download.pytorch.org/models/vgg16_bn-6c64b313.pth" to /root/.cache/torch/hub/checkpoints/vgg16_bn-6c64b313.pth
100%|██████████| 528M/528M [00:03<00:00, 184MB/s]

# Replacing the classifier(FCL) with the same architecture used during training
model_vgg.classifier = torch.nn.Sequential(
    torch.nn.Linear(512 * 7 * 7, 4096),
    torch.nn.ReLU(True),
    torch.nn.Dropout(p=0.5),
    torch.nn.Linear(4096, 4096),
    torch.nn.ReLU(True),
    torch.nn.Dropout(p=0.5),
    torch.nn.Linear(4096, 3)
)

# Load the saved weights of the best model
model_vgg.load_state_dict(torch.load("/content/drive/MyDrive/CV CW/vgg_finetuned_mask_classifier.pth"))
model_vgg.to(device)
model_vgg.eval()

# Transformations for faces processing
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

def MaskDetection(path):
    cap = cv2.VideoCapture(path) # Load the video

    frames = []
    while cap.isOpened():
        ret, frame = cap.read()

```